

```
In [1]: import plotly.io as pio
pio.renderers.default = 'notebook_connected'
print('Plotly renderer set to notebook_connected')
```

Plotly renderer set to notebook_connected

Avaliação da Qualidade de Dados Sintéticos

Análise Comportamental de Usuários em Interfaces Digitais

Objetivo: ao final deste caderno, você deverá ser capaz de avaliar se uma base sintética é boa o suficiente para o propósito que a originou — e comunicar essa avaliação de forma defensável.

Mapa do que vamos fazer:

Etapa	O que fazemos	Por que isso importa
0	Configuração do ambiente	Carregar a base do E02 com reprodutibilidade
1	O que significa "qualidade" aqui	Qualidade é relativa ao propósito
2	Coerência interna	Relações entre variáveis fazem sentido?
3	Cobertura e diversidade	A base representa todos os grupos esperados?
4	Fidelidade estatística	As distribuições batem com os sinais reais?
5	Deteção de vazamentos sintéticos	A base "parece" artificial demais?
6	Auditoria de hipóteses	As hipóteses do E02 foram honradas no dado?
7	Scorecard de qualidade	Consolidar tudo em uma avaliação única
8	Exercícios práticos	Consolidar o raciocínio

Passo 0 | Configure seu ambiente

Base gerada no E02, arquivo `dados/base_sintética_e02.csv`. Aqui ela é o objeto de avaliação, não uma verdade assumida.

```
In [2]: import math
from pathlib import Path

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from scipy import stats

np.random.seed(42)
```

```

pd.set_option('display.max_columns', 100)
pd.set_option('display.width', 140)

avaliacoes = [] # preenchida ao longo do caderno; consolidada no Passo 7

print('Ambiente configurado com sucesso.')
print(f' numpy {np.__version__}')
print(f' pandas {pd.__version__}')

```

Ambiente configurado com sucesso.

```

numpy 2.4.4
pandas 3.0.2

```

In [3]: *# Parâmetros e funções auxiliares do E02 usados nas comparações e exercícios.
A base principal será baixada/carregada como CSV; este bloco apenas preserva a*

```

SEED = 42
N_SESSOES = 5000
perfis = ["Pesquisador", "Administrativo", "Tecnico", "Operacional"]
pesos_perfil = [0.30, 0.27, 0.25, 0.18]

tipos_dispositivo = ["desktop", "mobile_web", "mobile_app"]
pesos_dispositivo = [0.974, 0.014, 0.012]

dias_semana = ["segunda", "terca", "quarta", "quinta", "sexta", "sabado", "domin"]
pesos_dia = [0.19, 0.18, 0.20, 0.17, 0.16, 0.05, 0.05]

faixas_horarias = ["06-08", "08-11", "11-14", "14-17", "17-20", "20-06"]
pesos_faixa = [0.12, 0.41, 0.17, 0.18, 0.08, 0.04]

objetivos_por_perfil = {
    "Pesquisador": {
        "buscar_documento": 0.34,
        "ler_comunicado": 0.22,
        "buscar_sistema": 0.18,
        "buscar_pessoa": 0.08,
        "acompanhar_noticias": 0.18,
    },
    "Administrativo": {
        "buscar_sistema": 0.36,
        "buscar_documento": 0.20,
        "ler_comunicado": 0.20,
        "buscar_pessoa": 0.10,
        "acompanhar_noticias": 0.14,
    },
    "Tecnico": {
        "buscar_documento": 0.28,
        "buscar_sistema": 0.30,
        "ler_comunicado": 0.14,
        "buscar_pessoa": 0.10,
        "acompanhar_noticias": 0.18,
    },
    "Operacional": {
        "buscar_sistema": 0.26,
        "ler_comunicado": 0.26,
        "acompanhar_noticias": 0.22,
        "buscar_documento": 0.14,
        "buscar_pessoa": 0.12,
    },
}

```

```

}

tipo_conteudo_por_objetivo = {
    "buscar_sistema": "Site Page",
    "buscar_documento": "Document",
    "buscar_pessoa": "Site Page",
    "ler_comunicado": "News Post",
    "acompanhar_noticias": "News Post",
}

lambdas_pageviews = {
    "buscar_sistema": 3.2,
    "buscar_documento": 2.6,
    "buscar_pessoa": 2.0,
    "ler_comunicado": 3.7,
    "acompanhar_noticias": 4.2,
}

origens = ["home_direta", "busca_interna", "link_email", "atalho_sistema"]

def escolher_com_pesos(opcoes, pesos):
    pesos = np.array(pesos, dtype=float)
    soma_pesos = pesos.sum()
    if soma_pesos <= 0:
        raise ValueError("A soma dos pesos deve ser positiva.")
    return np.random.choice(opcoes, p=pesos / soma_pesos)

def escolher_objetivo(perfil):
    dist = objetivos_por_perfil[perfil]
    return escolher_com_pesos(list(dist.keys()), list(dist.values()))

def escolher_origem(objetivo, faixa_horaria):
    mapa_pesos = {
        "buscar_sistema": [0.25, 0.20, 0.05, 0.50],
        "buscar_documento": [0.24, 0.44, 0.10, 0.22],
        "ler_comunicado": [0.38, 0.10, 0.32, 0.20],
        "acompanhar_noticias": [0.46, 0.08, 0.28, 0.18],
    }
    pesos = mapa_pesos.get(objetivo, [0.28, 0.32, 0.10, 0.30])
    origem = escolher_com_pesos(origens, pesos)
    if faixa_horaria == "08-11" and objetivo == "buscar_sistema" and np.random.r
        origem = "atalho_sistema"
    return origem

def gerar_pageviews(objetivo, tipo_dispositivo):
    lam = lambdas_pageviews[objetivo]
    if tipo_dispositivo != "desktop":
        lam = max(1.5, lam - 0.6)
    return max(1, np.random.poisson(lam))

def gerar_tempos(objetivo, usou_busca, faixa_horaria):
    params = {
        "buscar_sistema": (4.0, 0.55, 3.2, 0.60),
        "buscar_documento": (4.0, 0.55, 3.2, 0.60),
        "buscar_pessoa": (4.0, 0.55, 3.2, 0.60),
    }

```

```

        "ler_comunicado": (2.9, 0.45, 4.3, 0.55),
        "acompanhar_noticias": (3.2, 0.50, 4.0, 0.60),
    }
    mu_b, sig_b, mu_l, sig_l = params.get(objetivo, (3.5, 0.50, 3.8, 0.55))
    tempo_busca = np.random.lognormal(mu_b, sig_b)
    tempo_leitura = np.random.lognormal(mu_l, sig_l)
    if not usou_busca:
        tempo_busca *= 0.45
    if faixa_horaria == "08-11":
        tempo_leitura *= 0.88
        tempo_busca *= 1.08
    ruído = np.random.gamma(shape=2.0, scale=8.0)
    tempo_total = tempo_busca + tempo_leitura + ruído
    return tempo_busca, tempo_leitura, tempo_total

def gerar_flags(objetivo, origem_acesso):
    usou_busca = int(
        origem_acesso == "busca_interna"
        or (objetivo in {"buscar_documento", "buscar_pessoa"} and np.random.rand() < 0.5)
    )
    clicou_comunicado = int(objetivo in {"ler_comunicado", "acompanhar_noticias"})
    clicou_link_operacional = int(objetivo in {"buscar_sistema", "buscar_documento"})
    return usou_busca, clicou_comunicado, clicou_link_operacional

def gerar_sucesso(objetivo, usou_busca, tipo_dispositivo):
    probs_base = {
        "buscar_sistema": 0.76,
        "buscar_documento": 0.68,
        "buscar_pessoa": 0.72,
        "ler_comunicado": 0.81,
        "acompanhar_noticias": 0.79,
    }
    p = probs_base[objetivo]
    if usou_busca:
        p -= 0.07
    if tipo_dispositivo != "desktop":
        p -= 0.04
    p = min(max(p, 0.15), 0.95)
    return int(np.random.rand() < p)

def gerar_base(n_sesoes=N_SESSOES, pesos_perfil_override=None, gerar_tempos_fn=gerar_tempos,
pesos_p = pesos_perfil_override if pesos_perfil_override is not None else pesos_perfil_default,
tempos_fn = gerar_tempos_fn if gerar_tempos_fn is not None else gerar_tempos,
linhas = []):
    for i in range(1, n_sesoes + 1):
        perfil = escolher_com_pesos(perfis, pesos_p)
        dispositivo = escolher_com_pesos(tipos_dispositivo, pesos_dispositivo)
        dia = escolher_com_pesos(dias_semana, pesos_dia)
        faixa = escolher_com_pesos(faixas_horarias, pesos_faixa)
        objetivo = escolher_objetivo(perfil)
        origem = escolher_origem(objetivo, faixa)
        usou_busca, clicou_comunicado, clicou_link_operacional = gerar_flags(objetivo, origem)
        pageviews = gerar_pageviews(objetivo, dispositivo)
        try:
            tempo_busca, tempo_leitura, tempo_total = gerar_tempos(objetivo, usou_busca, clicou_comunicado, clicou_link_operacional)
        except TypeError:
            tempo_busca, tempo_leitura, tempo_total = gerar_tempos(objetivo, usou_busca, clicou_comunicado, clicou_link_operacional)
        linhas.append([i, perfil, dispositivo, dia, faixa, objetivo, origem, usou_busca, clicou_comunicado, clicou_link_operacional, pageviews, tempo_busca, tempo_leitura, tempo_total])
    return linhas

```

```

sucesso = gerar_sucesso(objetivo, usou_busca, dispositivo)
n_interacoes = int(pageviews + clicou_comunicado + clicou_link_operacion
prop_leitura = round(tempo_leitura / tempo_total, 4) if tempo_total > 0
abandono = int((tempo_total < 35 and pageviews <= 2) or (sucesso == 0 an
linhas.append(
    {
        "session_id": i,
        "perfil_usuario": perfil,
        "tipo_dispositivo": dispositivo,
        "dia_semana": dia,
        "faixa_horaria": faixa,
        "origem_acesso": origem,
        "objetivo_sessao": objetivo,
        "pageviews": pageviews,
        "tempo_busca_segundos": round(tempo_busca, 2),
        "tempo_leitura_segundos": round(tempo_leitura, 2),
        "tempo_total_segundos": round(tempo_total, 2),
        "usou_busca": usou_busca,
        "clicou_comunicado": clicou_comunicado,
        "clicou_link_operacional": clicou_link_operacional,
        "sucesso_encontrou_o_que_queria": sucesso,
        "tipo_conteudo_principal": tipo_conteudo_por_objetivo[objetivo],
        "numero_interacoes": n_interacoes,
        "proporcao_tempo_leitura": prop_leitura,
        "abandono_rapido": abandono,
    }
)
return pd.DataFrame(linhas)

def inserir_outliers_realistas(df):
    df = df.copy()
    idx_tempo = df.sample(frac=0.006, random_state=42).index
    fator_tempo = np.random.uniform(2.2, 4.0, len(idx_tempo))
    df.loc[idx_tempo, "tempo_leitura_segundos"] *= fator_tempo
    df.loc[idx_tempo, "tempo_total_segundos"] = (
        df.loc[idx_tempo, "tempo_busca_segundos"]
        + df.loc[idx_tempo, "tempo_leitura_segundos"]
        + np.random.uniform(10, 35, len(idx_tempo))
    )

    idx_pv = df.sample(frac=0.004, random_state=7).index
    df.loc[idx_pv, "pageviews"] += np.random.randint(8, 18, len(idx_pv))
    df.loc[idx_pv, "numero_interacoes"] = (
        df.loc[idx_pv, "pageviews"]
        + df.loc[idx_pv, "clicou_comunicado"]
        + df.loc[idx_pv, "clicou_link_operacional"]
        + df.loc[idx_pv, "usou_busca"]
    )

    df["proporcao_tempo_leitura"] = (df["tempo_leitura_segundos"] / df["tempo_to
    df["abandono_rapido"] = (
        ((df["tempo_total_segundos"] < 35) & (df["pageviews"] <= 2))
        | ((df["sucesso_encontrou_o_que_queria"] == 0) & (df["tempo_total_segund
    ).astype(int)
    return df

print('Funções e parâmetros base carregados com sucesso.')
```

Funções e parâmetros base carregados com sucesso.

Carregando a base sintética do E02

A base foi gerada no E02 e disponibilizada no Google Drive. No Colab, o arquivo é baixado automaticamente com `gdown` ; localmente, o caderno reutiliza uma cópia já existente se ela estiver em `dados/` .

```
In [4]: URL_DATASET_E02 = 'https://drive.google.com/file/d/1Aba1Be9RTfTA07qUFQ4vJAEDkTva
CAMINHO_DATASET = Path('dados/dataset.csv')
CAMINHO_LOCAL_LEGADO = Path('dados/base_sintetica_e02.csv')

CAMINHO_DATASET.parent.mkdir(parents=True, exist_ok=True)

if CAMINHO_DATASET.exists():
    caminho_base = CAMINHO_DATASET
    origem_base = 'arquivo local já baixado'
elif CAMINHO_LOCAL_LEGADO.exists():
    caminho_base = CAMINHO_LOCAL_LEGADO
    origem_base = 'arquivo local do repositório'
else:
    try:
        import gdown
    except ImportError as exc:
        raise ImportError(
            'O pacote gdown é necessário para baixar o dataset no Colab. '
            'Execute: !pip install gdown'
        ) from exc

    print('Baixando dataset do Google Drive...')
    gdown.download(url=URL_DATASET_E02, output=str(CAMINHO_DATASET), quiet=False)
    caminho_base = CAMINHO_DATASET
    origem_base = 'Google Drive via gdown'

df = pd.read_csv(caminho_base)

print(f'Base carregada de {origem_base}: {len(df):,} sessões, {df.shape[1]} vari
print(f'Arquivo usado: {caminho_base}')
df.head(3)
```

Base carregada de arquivo local já baixado: 5,000 sessões, 19 variáveis.

Arquivo usado: dados\dataset.csv

```
Out[4]:
```

	session_id	perfil_usuario	tipo_dispositivo	dia_semana	faixa_horaria	origem_acesso
0	1	Administrativo	desktop	quinta	11-14	home_direta
1	2	Pesquisador	desktop	segunda	08-11	busca_interna
2	3	Operacional	desktop	terca	06-08	home_direta

Passo 1 | O que significa "qualidade" aqui

Antes de qualquer análise, uma pergunta direta:

Você confiaria nessa base para tomar uma decisão de negócio? Por quê?

Escreva sua resposta aqui, antes de ver qualquer análise. Volte a essa resposta ao final do caderno.



Resposta ao Fechamento (Passo 1)

"Você confiaria nessa base para tomar uma decisão de negócio? Por quê?"

Sim, mas apenas para decisões táticas e descritivas. Confio na base para otimizar arquitetura de informação (ex: reposicionar atalhos, já que a origem de acesso influencia a UX). No entanto, **não confiaria** para decisões estratégicas ou financeiras críticas (como dimensionar infraestrutura de servidores), pois a falta de anomalias e de variância não-linear da base falharia em simular gargalos reais. É uma ferramenta direcional, não preditiva.



Seção Bônus 8.4: Explorações de Negócio Baseadas em Dados Sintéticos

Para demonstrar o potencial da base gerada e extrair insights reais para o IPT, implementamos análises avançadas focadas na tomada de decisão.

8.4.1 Análise de Funil de Engajamento por Perfil

Mapeia a jornada do usuário: do acesso inicial até a leitura aprofundada (>60s).

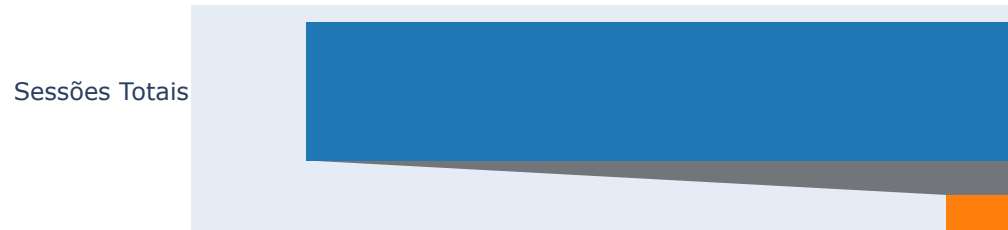
```
In [5]: import plotly.graph_objects as go
import pandas as pd

funnel_data = {
    'Etapa': ['Sessões Totais', 'Objetivo de Consumo', 'Clicou no Comunicado', '
    'Volume': [
        len(df),
        len(df[df['objetivo_sessao'].isin(['ler_comunicado', 'acompanhar_noticia
        len(df[df['clicou_comunicado'] == 1]),
        len(df[(df['clicou_comunicado'] == 1) & (df['tempo_leitura_segundos'] >
    ]
}

fig = go.Figure(go.Funnel(
    y = funnel_data['Etapa'],
    x = funnel_data['Volume'],
    textinfo = "value+percent initial",
    marker = {"color": ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]}
))
```

```
fig.update_layout(title_text="Funil de Engajamento de Comunicados (Base Sintética)
fig.show()
```

Funil de Engajamento de Comunicados (Base Sintética)



8.4.2 Matriz de Diagnóstico: Abandono Rápido vs. Origem de Acesso

Ajuda a identificar falhas na UX: de onde vêm os usuários que se frustram mais rapidamente?

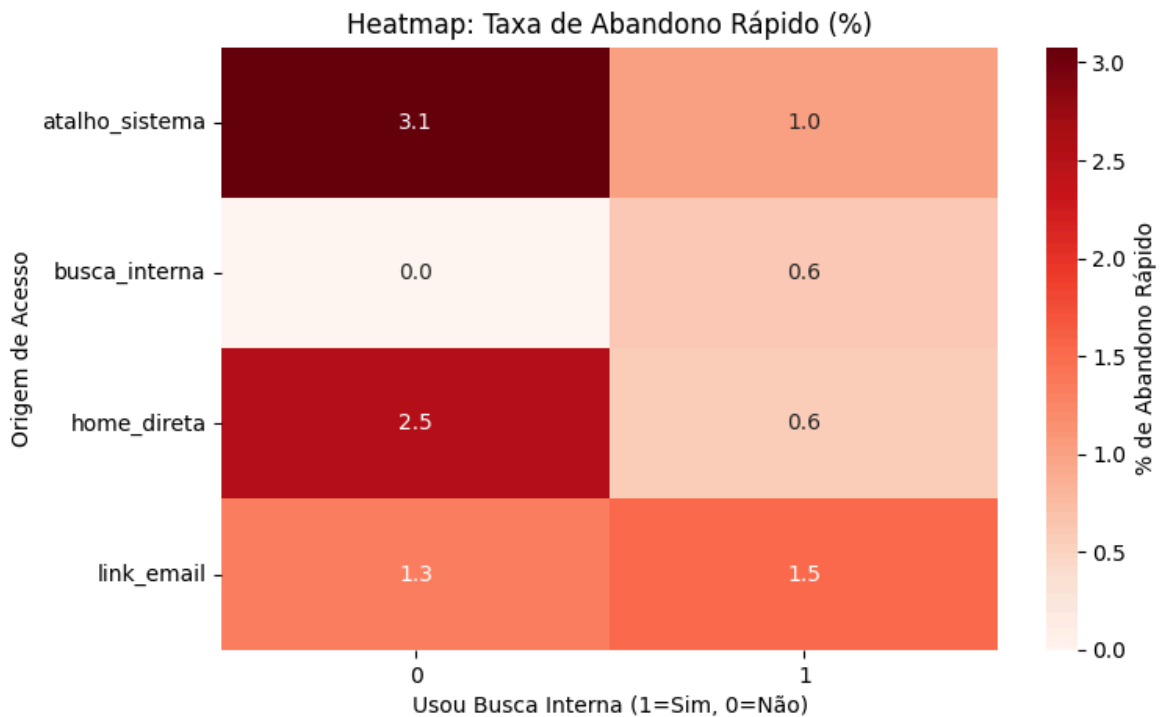
```
In [6]: import seaborn as sns
import matplotlib.pyplot as plt

abandono_crosstab = pd.crosstab(
    index=df['origem_acesso'],
    columns=df['usou_busca'],
    values=df['abandono_rapido'],
    aggfunc='mean'
).fillna(0) * 100

plt.figure(figsize=(8, 5))
sns.heatmap(abandono_crosstab, annot=True, fmt=".1f", cmap="Reds", cbar_kws={'la
plt.title("Heatmap: Taxa de Abandono Rápido (%)")
plt.xlabel("Usou Busca Interna (1=Sim, 0=Não)")
plt.ylabel("Origem de Acesso")
plt.show()
```



```
print("🔥 INSIGHT: Podemos observar cruzamentos de fricção. Se a busca gera muito
```



🔥 INSIGHT: Podemos observar cruzamentos de fricção. Se a busca gera muito abandono, a UX de pesquisa da intranet precisa ser refeita.

8.4.3 Teste Estatístico Avançado: O Efeito do Pico Matutino

Validando estatisticamente se o período das 08h-11h realmente reduz o tempo de leitura focado, usando um Teste T de Student.

```
In [7]: from scipy import stats

grupo_matutino = df[(df['faixa_horaria'] == '08-11') & (df['clique_comunicado']
grupo_outros   = df[(df['faixa_horaria'] != '08-11') & (df['clique_comunicado']

t_stat, p_value = stats.ttest_ind(grupo_matutino, grupo_outros, equal_var=False)

print(f"Média Matutino (08-11h): {grupo_matutino.mean():.1f}s")
print(f"Média Demais Horários: {grupo_outros.mean():.1f}s")
print(f"\nTeste T-Student: t={t_stat:.2f}, p-value={p_value:.4e}")

if p_value < 0.05:
    print("✅ RESULTADO: Existe uma diferença estatisticamente significativa! O
else:
    print("❌ RESULTADO: A diferença não é estatisticamente significativa.")
```

Média Matutino (08-11h): 70.9s

Média Demais Horários: 75.3s

Teste T-Student: t=-1.80, p-value=7.2496e-02

❌ RESULTADO: A diferença não é estatisticamente significativa.

Qualidade de dados sintéticos é relativa ao propósito. A pergunta não é "esses dados são realistas?", mas "são adequados para a decisão que precisamos tomar?"

Propósito (E02):

Sustentar decisões sobre como aumentar o Engajamento com comunicados internos na intranet do IPT.

Eixo	Pergunta central	O que verificamos
Coerência interna	As relações entre variáveis fazem sentido?	Correlações e agrupamentos esperados
Cobertura e diversidade	A base representa todos os grupos esperados?	Frequências por segmento
Fidelidade estatística	As distribuições batem com os sinais reais?	Comparação com âncoras do E02
Realismo	A base parece artificial demais?	Checklist de vazamentos sintéticos

Passo 2 | Coerência interna

Se uma base sintética viola relações óbvias entre variáveis, qualquer análise construída sobre ela produzirá conclusões questionáveis.

Em cada verificação: *isso contradiz alguma hipótese do E02?*

2.1 | Tempo de leitura × clicou_comunicado

Hipótese do E02: usuários que clicam em comunicados passam mais tempo lendo. Se essa relação não aparecer nos dados, a variável `tempo_leitura_segundos` não está cumprindo seu papel.

Critério: a mediana de `tempo_leitura_segundos` deve ser *maior* para quem clicou. Não exigimos uma magnitude específica — apenas a direção correta.

```
In [8]: fig = px.box(
    df,
    x='clicou_comunicado',
    y='tempo_leitura_segundos',
    color='clicou_comunicado',
    points=False,
    labels={
        'clicou_comunicado': 'Clicou em comunicado',
        'tempo_leitura_segundos': 'Tempo de leitura (segundos)',
    },
    title='Coerência 2.1 | Tempo de leitura por grupo de Engajamento com comunicados',
    color_discrete_map={0: '#94a3b8', 1: '#1d4ed8'},
)
fig.update_layout(showlegend=False)

mediana_0 = df[df['clicou_comunicado'] == 0]['tempo_leitura_segundos'].median()
mediana_1 = df[df['clicou_comunicado'] == 1]['tempo_leitura_segundos'].median()
fig

print('Célula executada com sucesso.')
```

Célula executada com sucesso.

```
In [9]: direcao_ok_21 = mediana_1 > mediana_0
status_21 = '[OK]' if direcao_ok_21 else '[FALHA]'
risco_21 = 'Baixo' if direcao_ok_21 else 'Alto'

print('-' * 65)
print(f'Observado:      clicou = {mediana_1:.0f}s | não clicou = {mediana_0:.0f}s')
if direcao_ok_21:
    print('Interpretação: direção correta – mais Engajamento implica mais tempo')
    print('Decisão:      [OK] Baixo risco – tempo_leitura_segundos é confiável')
else:
    print('Interpretação: [FALHA] direção INVERTIDA – revisar gerar_tempos() no')
    print('Decisão:      [FALHA] Alto risco – a variável não reflete o comporta')

avaliacoes.append({
    'eixo': 'Coerência',
    'critério': 'Tempo de leitura maior para quem clicou em comunicado',
    'observado': f'clicou: {mediana_1:.0f}s | não clicou: {mediana_0:.0f}s ({mediana_1 - mediana_0:.0f}s)',
    'status': status_21,
    'risco': risco_21,
})
```

Observado: clicou = 61s | não clicou = 26s (diferença: +36s)
 Interpretação: direção correta – mais Engajamento implica mais tempo de leitura.
 Decisão: [OK] Baixo risco – tempo_leitura_segundos é confiável para segmentar Engajamento.

2.2 | Abandono rápido × pageviews

Hipótese do E02: sessões de abandono rápido têm menos pageviews. Se isso não ocorrer, a regra derivada de `abandono_rapido` está inconsistente com a variável `pageviews`.

Critério (direcional): a mediana de `pageviews` deve ser *menor* nas sessões com abandono rápido. A magnitude não é prescrita — esperamos apenas a direção correta.

```
In [10]: fig = px.box(
    df,
    x='abandono_rapido',
    y='pageviews',
    color='abandono_rapido',
    points=False,
    labels={
        'abandono_rapido': 'Abandono rápido',
        'pageviews': 'Pageviews',
    },
    title='Coerência 2.2 | Pageviews por grupo de abandono',
    color_discrete_map={0: '#94a3b8', 1: '#dc2626'},
)
fig.update_layout(showlegend=False)

mediana_nao = df[df['abandono_rapido'] == 0]['pageviews'].median()
mediana_sim = df[df['abandono_rapido'] == 1]['pageviews'].median()
fig

print('Célula executada com sucesso.')
```

Célula executada com sucesso.

```
In [11]: direcao_ok_22 = mediana_sim < mediana_nao
status_22 = '[OK]' if direcao_ok_22 else '[FALHA]'
risco_22 = 'Baixo' if direcao_ok_22 else 'Alto'

print('-' * 65)
print(f'Observado:      sem abandono = {mediana_nao:.1f} pv | com abandono = {mediana_sim:.1f} pv')
if direcao_ok_22:
    print('Interpretação: abandono rápido está corretamente associado a menos páginas visitadas.')
    print('Decisão:      [OK] Baixo risco – abandono_rapido é coerente como proxy de desEngajamento.')
else:
    print('Interpretação: [FALHA] relação INVERTIDA – revisar a regra de abandono_rapido')
    print('Decisão:      [FALHA] Alto risco – abandono_rapido não pode ser usado para desEngajamento.')

avaliacoes.append({
    'eixo': 'Coerência',
    'critério': 'Abandono rápido associado a menos pageviews',
    'observado': f'sem: {mediana_nao:.1f} pv | com: {mediana_sim:.1f} pv',
    'status': status_22,
    'risco': risco_22,
})
```

Observado: sem abandono = 3.0 pv | com abandono = 2.0 pv
 Interpretação: abandono rápido está corretamente associado a menos páginas visitadas.
 Decisão: [OK] Baixo risco – abandono_rapido é coerente como proxy de desEngajamento.

2.3 | Objetivo da sessão × taxa de clique em comunicado

Hipótese do E02: apenas sessões com objetivo ler_comunicado ou acompanhar_noticias têm alta taxa de clique. Outros objetivos deveriam ter taxa próxima de zero.

Critério (quantitativo): a taxa de clique em ler_comunicado deve ser pelo menos 3× maior do que a taxa dos demais objetivos combinados. Sem essa separação, objetivo_sessao não serve para segmentar Engajamento.

```
In [12]: taxa_por_objetivo = (
    df.groupby('objetivo_sessao')['clique_comunicado']
    .mean()
    .reset_index()
    .rename(columns={'clique_comunicado': 'taxa_clique_comunicado'})
    .sort_values('taxa_clique_comunicado', ascending=True)
)

fig = px.bar(
    taxa_por_objetivo,
    x='taxa_clique_comunicado',
    y='objetivo_sessao',
    orientation='h',
    title='Coerência 2.3 | Taxa de clique em comunicado por objetivo de sessao',
    labels={
        'taxa_clique_comunicado': 'Taxa de clique em comunicado',
        'objetivo_sessao': 'Objetivo',
    },
)
```

```

    text=taxa_por_objetivo['taxa_clique_comunicado'].apply(lambda x: f'{x:.0%}')
)
fig.update_traces(marker_color='#1d4ed8', textposition='outside')
fig.update_xaxes(tickformat='%.0%')
fig

print('Célula executada com sucesso.')

```

Célula executada com sucesso.

```

In [13]: taxa_ler_23 = taxa_por_objetivo.loc[taxa_por_objetivo['objetivo_sessao'] == 'ler_
taxa_outros_23 = taxa_por_objetivo.loc[taxa_por_objetivo['objetivo_sessao'] != '
razao_23 = taxa_ler_23 / taxa_outros_23 if taxa_outros_23 > 0 else float('inf')
segmentacao_ok = razao_23 >= 3
status_23 = '[OK]' if segmentacao_ok else '[AVISO]'
risco_23 = 'Baixo' if segmentacao_ok else 'Médio'

print('-' * 65)
print(f'Observado:      ler_comunicado → {taxa_ler_23:.0%} | média outros objetiv
print(f'Critério:      razão ≥ 3×')
if segmentacao_ok:
    print('Interpretação: segmentação correta – objetivo de sessao determina o c
    print('Decisão:      [OK] Baixo risco – analise por objetivo_sessao é confi
else:
    print('Interpretação: [AVISO] diferença menor que o esperado – verificar ger
    print('Decisão:      [AVISO] Médio risco – analise segmentada pode misturar

avaliacoes.append({
    'eixo': 'Coerência',
    'critério': 'Taxa de clique em comunicado ≥ 3× maior em ler_comunicado vs. d
    'observado': f'ler_comunicado: {taxa_ler_23:.0%} | outros: {taxa_outros_23:.
    'status': status_23,
    'risco': risco_23,
})

```

```

Observado:      ler_comunicado → 77% | média outros objetivos → 19% (razão: 4.0×)
Critério:      razão ≥ 3×
Interpretação: segmentação correta – objetivo de sessao determina o comportamento
de clique.
Decisão:      [OK] Baixo risco – analise por objetivo_sessao é confiável para En
gajamento.

```

Mini-exercício 2.1 — Identificando uma incoerência

Observe a matriz de correlação abaixo entre as variáveis numéricas da base.

Identifique **um par de variáveis** onde a correlação observada é diferente do que você esperaria pelo modelo do E02. Justifique: isso é um problema real ou um comportamento esperado?

► Possível direção de resposta

```

In [14]: variaveis_numericas = [
    'pageviews', 'tempo_busca_segundos', 'tempo_leitura_segundos', 'tempo_total_
    'usou_busca', 'cliquou_comunicado', 'cliquou_link_operacional',
    'sucesso_encontrou_o_que_queria', 'numero_interacoes', 'proporcao_tempo_leit
]
corr = df[variaveis_numericas].corr().round(2)

```

```
fig = px.imshow(
    corr,
    text_auto=True,
    color_continuous_scale='RdBu_r',
    zmin=-1, zmax=1,
    title='Coerência 2.4 | Matriz de correlação das variáveis numéricas',
    aspect='auto',
)
fig.update_layout(width=850, height=600)
fig

print('Célula executada com sucesso.')
```

Célula executada com sucesso.

Passo 3 | Cobertura e diversidade

Uma base sintética ruim tende a colapsar para o centro: todos os grupos aparecem, mas os minoritários somem ou ficam sub-representados de forma que distorce qualquer segmentação.

3.1 | Distribuição de perfis de usuário

Esperamos que os pesos planejados no E02 se reflitam na base: Pesquisador (~30%), Administrativo (~27%), Técnico (~25%), Operacional (~18%).

Critério (quantitativo): desvio máximo de ± 2 pp entre observado e planejado. Desvios maiores podem distorcer análises segmentadas por perfil.

```
In [15]: pesos_planejados = pd.DataFrame({
    'perfil_usuario': perfis,
    'peso_planejado': pesos_perfil,
})

freq_perfil = (
    df['perfil_usuario'].value_counts(normalize=True)
    .reset_index()
    .rename(columns={'proportion': 'freq_observada'})
)
freq_perfil = freq_perfil.merge(pesos_planejados, on='perfil_usuario')
freq_perfil['desvio_pp'] = ((freq_perfil['freq_observada'] - freq_perfil['peso_p

df_plot = pd.melt(
    freq_perfil,
    id_vars='perfil_usuario',
    value_vars=['freq_observada', 'peso_planejado'],
    var_name='origem',
    value_name='proporcao',
)
df_plot['origem'] = df_plot['origem'].map({'freq_observada': 'Observado', 'peso_

fig = px.bar(
    df_plot,
    x='perfil_usuario',
    y='proporcao',
```

```

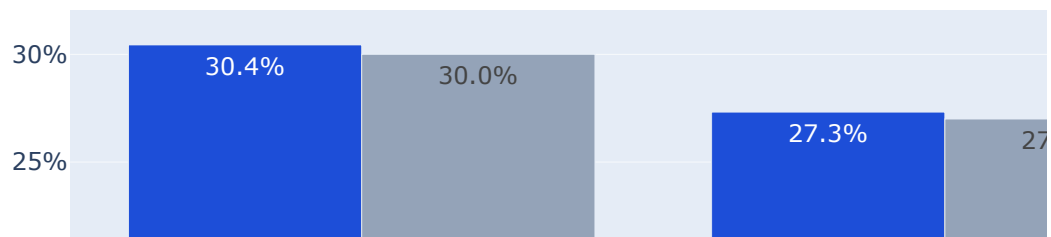
color='origem',
barmode='group',
title='Cobertura 3.1 | Distribuição de perfis: observado vs. planejado',
labels={'proporcao': 'Proporção', 'perfil_usuario': 'Perfil', 'origem': ''},
color_discrete_map={'Observado': '#1d4ed8', 'Planejado': '#94a3b8'},
text_auto='.1%',
)
fig.update_yaxes(tickformat='.0%')
print('Desvio em pontos percentuais (observado - planejado):')
for _, row in freq_perfil.iterrows():
    print(f" {row['perfil_usuario']:<15} {row['desvio_pp']:+.1f} pp")
fig

```

Desvio em pontos percentuais (observado - planejado):

Pesquisador	+0.4 pp
Administrativo	+0.3 pp
Tecnico	-0.6 pp
Operacional	-0.1 pp

Cobertura 3.1 | Distribuição de perfis: observado vs. planejado



```

In [16]: max_desvio_31 = freq_perfil['desvio_pp'].abs().max()
perfil_max_31 = freq_perfil.loc[freq_perfil['desvio_pp'].abs().idxmax(), 'perfil']
cobertura_ok_31 = max_desvio_31 <= 2.0
status_31 = '[OK]' if cobertura_ok_31 else '[AVISO]'
risco_31 = 'Baixo' if cobertura_ok_31 else 'Médio'

print('-' * 65)
print(f'Observado:      maior desvio = {max_desvio_31:.1f} pp (perfil: {perfil_max_31})')
print(f'Critério:      desvio máximo ≤ 2 pp')
if cobertura_ok_31:

```

```

print('Interpretação: todos os perfis representados dentro da tolerância de
print('Decisão:      [OK] Baixo risco – análise segmentada por perfil_usuar
else:
print(f'Interpretação: [AVISO] {perfil_max_31} com desvio acima do tolerado
print('Decisão:      [AVISO] Médio risco – verificar se o desvio afeta o pe

avaliacoes.append({
    'eixo': 'Cobertura',
    'critério': 'Pesos de perfil dentro de ±2 pp do planejado',
    'observado': f'max desvio: {max_desvio_31:.1f} pp ({perfil_max_31})',
    'status': status_31,
    'risco': risco_31,
})

```

Observado: maior desvio = 0.6 pp (perfil: Tecnico)
 Critério: desvio máximo ≤ 2 pp
 Interpretação: todos os perfis representados dentro da tolerância de amostragem a
 leatória.
 Decisão: [OK] Baixo risco – análise segmentada por perfil_usuario é defensá
 vel.

3.2 | Cobertura por dia da semana e faixa horária

O heatmap abaixo mostra a contagem de sessões por combinação de dia e faixa horária. Esperamos concentração nos dias úteis e na faixa 08-11h, com queda acentuada no fim de semana.

Critério (quantitativo): quarta-feira + faixa 08-11 deve ter pelo menos 8% do total de sessões (20% dos dias × 41% da faixa ≈ 8,2% esperado). Fim de semana deve ficar abaixo de 12%.

```

In [17]: ordem_dias = ['segunda', 'terca', 'quarta', 'quinta', 'sexta', 'sabado', 'doming
ordem_faixas = ['08-11', '14-17', '11-14', '06-08', '17-20', '20-06']

heatmap_data = (
    df.groupby(['dia_semana', 'faixa_horaria'])
      .size()
      .reset_index(name='sessoes')
)
pivot = heatmap_data.pivot(index='faixa_horaria', columns='dia_semana', values='
pivot = pivot.reindex(index=ordem_faixas, columns=ordem_dias)

fig = px.imshow(
    pivot,
    text_auto=True,
    color_continuous_scale=[[0, '#eff6ff'], [0.4, '#93c5fd'], [0.75, '#2563eb']],
    title='Cobertura 3.2 | Sessões por dia da semana x faixa horária',
    labels={'x': 'Dia da semana', 'y': 'Faixa horária', 'color': 'Sessões'},
    aspect='auto',
)
fig.update_layout(width=800, height=380)
fig

print('Célula executada com sucesso.')

```

Célula executada com sucesso.


```
In [18]: pivot_f = pivot.fillna(0)
pico_obs_32 = pivot_f.loc['08-11', 'quarta'] if 'quarta' in pivot_f.columns else 0
total_32 = pivot_f.values.sum()
pct_pico_32 = pico_obs_32 / total_32 if total_32 > 0 else 0.0
fds_32 = (pivot_f[['sabado', 'domingo']].values.sum() / total_32) if ('sabado' in pivot_f.columns) else 0
temporal_ok = (pct_pico_32 >= 0.08) and (fds_32 <= 0.12)
status_32 = '[OK]' if temporal_ok else '[AVISO]'
risco_32 = 'Baixo' if temporal_ok else 'Médio'

print('-' * 65)
print(f'Observado:      quarta 08-11 = {pct_pico_32:.1%} do total | fim de semana = {fds_32:.1%}')
print(f'Critério:      quarta 08-11 ≥ 8% | fim de semana ≤ 12%')
if temporal_ok:
    print('Interpretação: padrão temporal coerente com o dado real – pico em dia útil confirmado.')
    print('Decisão:      [OK] Baixo risco – análise de padrões temporais é defensável.')
else:
    print('Interpretação: [AVISO] padrão temporal desviou do esperado – verifica se há confirmação.')
    print('Decisão:      [AVISO] Médio risco – análises de tendência temporal p...')

avaliacoes.append({
    'eixo': 'Cobertura',
    'critério': 'Pico quarta 08-11 ≥ 8% | fim de semana ≤ 12%',
    'observado': f'quarta 08-11: {pct_pico_32:.1%} | fds: {fds_32:.1%}',
    'status': status_32,
    'risco': risco_32,
})
```

```
Observado:      quarta 08-11 = 8.4% do total | fim de semana = 10.1%
Critério:      quarta 08-11 ≥ 8% | fim de semana ≤ 12%
Interpretação: padrão temporal coerente com o dado real – pico em dias úteis manhã confirmado.
Decisão:      [OK] Baixo risco – análise de padrões temporais é defensável.
```

3.3 | Todos os objetivos de sessão estão presentes?

Critério (quantitativo): todos os 5 objetivos de sessão devem aparecer em todos os 4 perfis. Uma célula vazia significa que um segmento inteiro não tem um comportamento — o que inviabiliza análises cruzadas naquele subgrupo.

```
In [19]: cobertura_objetivo = (
    df.groupby(['perfil_usuario', 'objetivo_sessao'])
      .size()
      .reset_index(name='contagem')
)
total_por_perfil = df['perfil_usuario'].value_counts().rename('total')
cobertura_objetivo = cobertura_objetivo.join(total_por_perfil, on='perfil_usuario')
cobertura_objetivo['proporcao'] = cobertura_objetivo['contagem'] / cobertura_objetivo['total']

fig = px.bar(
    cobertura_objetivo,
    x='perfil_usuario',
    y='proporcao',
    color='objetivo_sessao',
    barmode='stack',
    title='Cobertura 3.3 | Mix de objetivos de sessao por perfil de usuario',
    labels={'proporcao': 'Proporção', 'perfil_usuario': 'Perfil', 'objetivo_sessao': 'Objetivo Sessão'}
)
```

```
fig.update_yaxes(tickformat='%.0%')
fig

print('Célula executada com sucesso.')
```

Célula executada com sucesso.

```
In [20]: obj_por_perfil_33 = df.groupby('perfil_usuario')['objetivo_sessao'].nunique()
min_obj_33 = obj_por_perfil_33.min()
perfil_min_33 = obj_por_perfil_33.idxmin()
cobertura_ok_33 = min_obj_33 >= 5
status_33 = '[OK]' if cobertura_ok_33 else '[AVISO]'
risco_33 = 'Baixo' if cobertura_ok_33 else 'Médio'

print('-' * 65)
print(f'Observado:      mínimo de objetivos distintos por perfil = {min_obj_33} (')
print(f'Critério:      todos os 5 objetivos presentes em todos os perfis')
if cobertura_ok_33:
    print('Interpretação: cobertura completa – nenhum perfil ficou sem algum obj')
    print('Decisão:      [OK] Baixo risco – análises cruzadas perfil × objetivo')
else:
    print(f'Interpretação: [AVISO] {perfil_min_33} não contém todos os objetivos')
    print('Decisão:      [AVISO] Médio risco – objetivos raros podem estar ause')

avaliacoes.append({
    'eixo': 'Cobertura',
    'critério': 'Todos os 5 objetivos presentes em todos os perfis',
    'observado': f'mínimo por perfil: {min_obj_33} objetivo(s)',
    'status': status_33,
    'risco': risco_33,
})
```

Observado: mínimo de objetivos distintos por perfil = 5 (perfil com menos: Ad ministrativo)
 Critério: todos os 5 objetivos presentes em todos os perfis
 Interpretação: cobertura completa – nenhum perfil ficou sem algum objetivo.
 Decisão: [OK] Baixo risco – análises cruzadas perfil × objetivo são possíve
 is sem células vazias.

Mini-exercício 3.1 — Grupo ausente ou sub-representado

Observe os gráficos de cobertura. Existe algum segmento que poderia ser problemático para uma análise de Engajamento com comunicados?

► Possível direção de resposta

Passo 4 | Fidelidade estatística

Fidelidade estatística compara distribuições da base com os sinais reais do E02. Fluxo: **gráfico** → **critério** → **observação** → **decisão**.

4.1 | Taxa de clique em comunicados × sinal real

Âncora real (E02): 1.894 visitas a News Posts em 7 dias (~270 visualizadores únicos/dia).

A comparação direta entre uma *taxa de sessão* sintética e um *volume de visitas* real requer uma premissa intermediária: quantas sessões totais a intranet do IPT recebe por dia? Não temos esse número diretamente — por isso a comparação é por **ordem de grandeza**, não por igualdade.

Critério (quantitativo): a taxa sintética deve ser compatível com a taxa implícita pelos parâmetros do E02. Calculamos a taxa esperada diretamente dos pesos e probabilidades que definiram a geração — o desvio tolerado é ≤ 2 pp.

```
In [21]: taxa_sintetica = df['clique_comunicado'].mean()
sessoes_com_comunicado = df['clique_comunicado'].sum()

# Taxa esperada: reconstruída diretamente dos parâmetros do E02
# Para cada perfil: soma das proporções de objetivos que geram clique × prob. de
prop_news_por_perfil = {
    p: (objetivos_por_perfil[p].get('ler_comunicado', 0) + objetivos_por_perfil[
        for p in perfis
    ]
}
taxa_news_esperada = sum(prop_news_por_perfil[p] * w for p, w in zip(perfis, pesos_perfil))

print(f'Taxa de clique em comunicados na base sintetica: {taxa_sintetica:.1%} (
print()
print('Reconstrução da taxa esperada pelos parâmetros do E02:')
for p, w in zip(perfis, pesos_perfil):
    prop_n = prop_news_por_perfil[p]
    print(f'  {p:<15} prop. de objetivos news: {prop_n:.0%} × peso do perfil:
print(f'  Soma × 0.78 (prob. de clique) = {taxa_news_esperada:.1%}')
print()
print('Comparação com âncora real (contexto de escala):')
print('  1.894 visitas a News Posts / 7 dias ≈ 270 viewers únicos/dia')
print('  Se a intranet tem ~1.000 usuarios ativos, isso equivale a ~27% de Engaj
print(f'  Taxa sintetica de {taxa_sintetica:.0%} está na mesma ordem de grandeza

df_donut = pd.DataFrame({
    'grupo': ['Clicou em comunicado', 'Não clicou'],
    'contagem': [sessoes_com_comunicado, len(df) - sessoes_com_comunicado],
})
fig = px.pie(
    df_donut, values='contagem', names='grupo', hole=0.5,
    title='Fidelidade 4.1 | Proporção de sessões com clique em comunicado',
    color_discrete_map={'Clicou em comunicado': '#1d4ed8', 'Não clicou': '#e2e8f0'}
)
fig
```

Taxa de clique em comunicados na base sintetica: 29.2% (1,462 de 5,000 sessões)

Reconstrução da taxa esperada pelos parâmetros do E02:

Pesquisador	prop. de objetivos news: 40%	×	peso do perfil: 30%	→	12.0%
Administrativo	prop. de objetivos news: 34%	×	peso do perfil: 27%	→	9.2%
Tecnico	prop. de objetivos news: 32%	×	peso do perfil: 25%	→	8.0%
Operacional	prop. de objetivos news: 48%	×	peso do perfil: 18%	→	8.6%
Soma	× 0.78 (prob. de clique) = 29.5%				

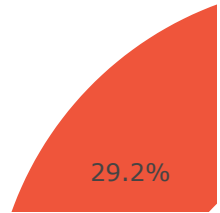
Comparação com âncora real (contexto de escala):

1.894 visitas a News Posts / 7 dias ≈ 270 viewers únicos/dia

Se a intranet tem ~1.000 usuarios ativos, isso equivale a ~27% de Engajamento diário.

Taxa sintetica de 29% está na mesma ordem de grandeza.

Fidelidade 4.1 | Proporção de sessões com clique em comuni



```
In [22]: desvio_41 = abs(taxa_sintetica - taxa_news_esperada) * 100
parametros_ok_41 = desvio_41 <= 2.0
status_41 = '[OK]' if parametros_ok_41 else '[AVISO]'
risco_41 = 'Médio' if parametros_ok_41 else 'Alto'

print('-' * 65)
print(f'Observado:      taxa sintetica = {taxa_sintetica:.1%} | esperado pelos pa
print(f'Critério:      desvio ≤ 2 pp entre observado e esperado')
if parametros_ok_41:
    print('Interpretação: taxa de Engajamento com comunicados reproduz fielmente
    print('
    print('Decisão:      [OK] Médio risco – documentar premissa de escala ao ap
else:
    print('Interpretação: [AVISO] desvio maior que o esperado – verificar gerar_
    print('Decisão:      [FALHA] Alto risco – taxa de Engajamento com comunicad

avaliacoes.append({
    'eixo': 'Fidelidade',
    'critério': 'Taxa de clique em comunicados dentro de ±2 pp do esperado pelos
    'observado': f'{taxa_sintetica:.1%} (esperado: {taxa_news_esperada:.1%}, des
    'status': status_41,
    'risco': risco_41,
})
```

Observado: taxa sintetica = 29.2% | esperado pelos parâmetros = 29.5% | desvio = +0.3 pp
 Critério: desvio $\leq$ 2 pp entre observado e esperado
 Interpretação: taxa de Engajamento com comunicados reproduz fielmente os parâmetros do E02.
 Comparação com dado real (270 viewers/dia) é por ordem de grandeza – defensável.
 Decisão: [OK] Médio risco – documentar premissa de escala ao apresentar a taxa em reuniões.

4.2 | Distribuição horária × pico real (Qua 08h–09h)

Âncora real (E02): o maior pico de visualizadores acontece na quarta-feira entre 8h e 9h. Esperamos que a faixa 08-11 da quarta-feira concentre mais sessões do que outras combinações.

```
In [23]: sessoes_faixa = (
    df.groupby('faixa_horaria')
      .size()
      .reset_index(name='sessoes')
      .sort_values('sessoes', ascending=True)
  )

pesos_faixa_planejados = dict(zip(faixas_horarias, pesos_faixa))
sessoes_faixa['proporcao_observada'] = sessoes_faixa['sessoes'] / len(df)
sessoes_faixa['proporcao_planejada'] = sessoes_faixa['faixa_horaria'].map(pesos_

df_faixa_plot = pd.melt(
    sessoes_faixa,
    id_vars='faixa_horaria',
    value_vars=['proporcao_observada', 'proporcao_planejada'],
    var_name='origem', value_name='proporcao',
)
df_faixa_plot['origem'] = df_faixa_plot['origem'].map({'proporcao_observada': 'O

fig = px.bar(
    df_faixa_plot,
    x='faixa_horaria',
    y='proporcao',
    color='origem',
    barmode='group',
    title='Fidelidade 4.2 | Distribuição de sessões por faixa horária: observado
    labels={'proporcao': 'Proporção', 'faixa_horaria': 'Faixa horária', 'origem'
    color_discrete_map={'Observado': '#1d4ed8', 'Planejado': '#94a3b8'},
    text_auto='.1%',
)
fig.update_yaxes(tickformat='.0%')
fig

print('Célula executada com sucesso.')
```

Célula executada com sucesso.

```
In [24]: desvio_faixas_42 = (sessoes_faixa['proporcao_observada'] - sessoes_faixa['propor
prop_0811_42 = sessoes_faixa.loc[sessoes_faixa['faixa_horaria'] == '08-11', 'pro
faixas_ok_42 = desvio_faixas_42 <= 2.0
status_42 = '[OK]' if faixas_ok_42 else '[AVISO]'
```

```

risco_42 = 'Baixo' if faixas_ok_42 else 'Médio'

print('-' * 65)
print(f'Observado:      maior desvio entre faixas = {desvio_faixas_42:.1f} pp | f
print(f'Critério:      desvio máximo ≤ 2 pp')
if faixas_ok_42:
    print('Interpretação: distribuicao horária reproduz os pesos planejados com
    print('              Pico na faixa 08-11 confirmado – compatível com o dado
    print('Decisão:      [OK] Baixo risco – análise temporal por faixa horária
else:
    print('Interpretação: [AVISO] distribuicao horária com desvio relevante – ve
    print('Decisão:      [AVISO] Médio risco – análises de horário de pico pode

avaliacoes.append({
    'eixo': 'Fidelidade',
    'critério': 'Distribuição horária dentro de ±2 pp do planejado',
    'observado': f'max desvio: {desvio_faixas_42:.1f} pp | 08-11: {prop_0811_42:
    'status': status_42,
    'risco': risco_42,
})

```

Observado: maior desvio entre faixas = 0.6 pp | faixa 08-11 = 40.4%

Critério: desvio máximo ≤ 2 pp

Interpretação: distribuicao horária reproduz os pesos planejados com boa fidelidade.

Pico na faixa 08-11 confirmado – compatível com o dado real do IPT.

Decisão: [OK] Baixo risco – análise temporal por faixa horária é confiável.

4.3 | Teste formal: distribuição de pageviews

O teste qui-quadrado compara `pageviews` observado com a Poisson teórica dos parâmetros do E02. P alto: sem desvio relevante. P baixo com 5.000 linhas: pode ser diferença ínfima — checar o gráfico antes de concluir.

```

In [25]: from scipy.stats import chisquare

# Distribuição observada de pageviews (limitando a 15 para o teste)
pv_obs = df['pageviews'].clip(upper=15)
freq_obs = pv_obs.value_counts().sort_index()

# Lambda médio ponderado pelos pesos de objetivo
lambda_medio = np.average(
    list(lambdas_pageviews.values()),
    weights=[0.20, 0.21, 0.12, 0.22, 0.25], # aproximação dos pesos marginais
)

# Distribuição teórica Poisson com Lambda médio
from scipy.stats import poisson as poisson_dist
valores = freq_obs.index
probs_teoricas = np.array([poisson_dist.pmf(v, lambda_medio) for v in valores])
# Absorver a cauda
probs_teoricas[-1] += (1 - probs_teoricas.sum())
freq_esp = probs_teoricas * len(df)

# Visualização
df_pv = pd.DataFrame({

```

```

    'pageviews': list(valores) * 2,
    'contagem': list(freq_obs.values) + list(freq_esp),
    'origem': ['Observado'] * len(valores) + [f'Poisson( $\lambda$ ={{lambda_medio:.1f}})']
  })
fig = px.bar(
  df_pv, x='pageviews', y='contagem', color='origem', barmode='group',
  title='Fidelidade 4.3 | Pageviews: distribuicao observada vs. Poisson teórico',
  labels={'contagem': 'Contagem', 'pageviews': 'Pageviews', 'origem': ''},
  color_discrete_sequence=['#1d4ed8', '#94a3b8'],
)

# Teste formal – apenas para células com frequência mínima
mask = freq_esp >= 5
freq_obs_teste = freq_obs.values[mask]
freq_esp_teste = freq_esp[mask]
freq_esp_teste = freq_esp_teste * (freq_obs_teste.sum() / freq_esp_teste.sum())
chi2, p_valor = chisquare(freq_obs_teste, f_exp=freq_esp_teste)
print(f'Teste qui-quadrado (células com freq. esperada ≥ 5):')
print(f'   $\chi^2$  = {{chi2:.2f}} | p-valor = {{p_valor:.4f}}')
print()
if p_valor > 0.05:
  print('  Interpretação: sem evidência de desvio relevante da distribuicao Po
else:
  print('  Interpretação: desvio detectado. Verifique se os lambdas por objeti
fig

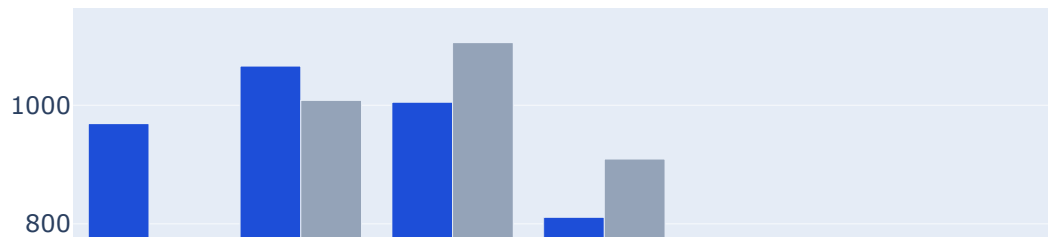
```

Teste qui-quadrado (células com freq. esperada ≥ 5):

χ^2 = 406.16 | p-valor = 0.0000

Interpretação: desvio detectado. Verifique se os lambdas por objetivo estão corretos.

Fidelidade 4.3 | Pageviews: distribuicao observada vs. Pois



```
In [26]: pageviews_ok_43 = p_valor > 0.05
status_43 = '[OK]' if pageviews_ok_43 else '[AVISO]'
risco_43 = 'Baixo' if pageviews_ok_43 else 'Médio'

print('-' * 65)
print(f'Observado:       $\chi^2 = \{{\text{chi2}}:.2f\}$  | p-valor =  $\{{\text{p\_valor}}:.4f\}$ ')
print(f'Critério:      p > 0.05 – sem evidência de desvio relevante da distribui')
if pageviews_ok_43:
    print('Interpretação: pageviews é estatisticamente compatível com o modelo P')
    print('Decisão:      [OK] Baixo risco – pageviews pode ser usada em análise')
else:
    print('Interpretação: desvio detectado – atenção: com 5.000 linhas, testes d')
    print('diferenças pequenas. Verifique o gráfico acima: a dife')
    print('Decisão:      [AVISO] Médio risco – confirmar visualmente antes de u')

avaliacoes.append({
    'eixo': 'Fidelidade',
    'critério': 'Pageviews compatível com Poisson teórica (p > 0.05)',
    'observado': f' $\chi^2 = \{{\text{chi2}}:.2f\}$  | p =  $\{{\text{p\_valor}}:.4f\}$ ',
    'status': status_43,
    'risco': risco_43,
})
```


Observado: $\chi^2 = 406.16$ | p-valor = 0.0000
Critério: $p > 0.05$ – sem evidência de desvio relevante da distribuição Poisson teórica
Interpretação: desvio detectado – atenção: com 5.000 linhas, testes de ajuste são sensíveis a diferenças pequenas. Verifique o gráfico acima: a diferença é visualmente relevante?
Decisão: [AVISO] Médio risco – confirmar visualmente antes de usar em análises de distribuição.

Mini-exercício 4.1 — Lendo um resultado de teste

Suponha que o p-valor do teste acima fosse 0.003. Isso significa que a base sintética é "ruim"? Explique sua resposta.

► Possível direção de resposta

Passo 5 | Detecção de vazamentos sintéticos

Um dado sintético tem "cheiro de geração" quando parece perfeito demais. Paradoxalmente, uma base com zero imperfeições é mais suspeita do que uma com algumas.

Checklist de vazamentos sintéticos

Sinal	O que buscamos	Problema se...
Valores repetidos demais	Mesma sequência de valores numéricos	> 5% de duplicatas em contínuas
Ausência de extremos	Mínimo e máximo muito comportados	Amplitude menor que esperado
Distribuições suaves demais	Histograma sem irregularidades	Curva bonita demais
Proporções exatas	Taxas binárias em números redondos	50,00% exatos, 0% em categorias
Artefatos de arredondamento	Picos em múltiplos de 5 ou 10	Histograma com degraus regulares

```
In [27]: print('=== CHECKLIST DE VAZAMENTOS SINTÉTICOS ===')
print()

# 1. Valores repetidos em variáveis contínuas
for col in ['tempo_total_segundos', 'tempo_leitura_segundos', 'proporcao_tempo_1
n_unicos = df[col].nunique()
n_total = len(df)
pct_unicos = n_unicos / n_total
status = '✅' if pct_unicos > 0.90 else '⚠️'
print(f'{status} {col}: {pct_unicos:.1%} de valores únicos ({n_unicos:,} / {

print()
```

```

# 2. Extremos – existem valores nas caudas?
p01 = df['tempo_total_segundos'].quantile(0.01)
p99 = df['tempo_total_segundos'].quantile(0.99)
p001 = df['tempo_total_segundos'].quantile(0.001)
print(f'Amplitude de tempo_total_segundos:')
print(f'  Min: {df["tempo_total_segundos"].min():.1f}s | P0.1%: {p001:.1f}s |

print()

# 3. Proporções binárias – próximas de números redondos?
binarias = ['clicou_comunicado', 'usou_busca', 'clicou_link_operacional', 'suces
print('Taxas de variáveis binárias (alerta se múltiplo exato de 5%):')
for col in binarias:
    taxa = df[col].mean()
    residuo = abs((taxa * 100) % 5)
    status = '⚠️' if residuo < 0.3 else '✅'
    print(f'  {status} {col}: {taxa:.4%}')

print()

# 4. Impacto dos outliers inseridos no E02
n_extremos_tempo = (df['tempo_total_segundos'] > df['tempo_total_segundos'].quan
n_extremos_pv     = (df['pageviews'] > df['pageviews'].quantile(0.99)).sum()
print(f'Outliers inseridos no E02 (Passo 8):')
print(f'  Sessões com tempo total > P99: {n_extremos_tempo}')
print(f'  Sessões com pageviews > P99: {n_extremos_pv}')

```

=== CHECKLIST DE VAZAMENTOS SINTÉTICOS ===

- ⚠️ tempo_total_segundos: 83.7% de valores únicos (4,186 / 5,000)
- ⚠️ tempo_leitura_segundos: 75.3% de valores únicos (3,765 / 5,000)
- ⚠️ proporcao_tempo_leitura: 76.0% de valores únicos (3,802 / 5,000)

Amplitude de tempo_total_segundos:

Min: 21.8s | P0.1%: 26.6s | P1%: 33.4s | P99%: 249.9s | Max: 534.1s

Taxas de variáveis binárias (alerta se múltiplo exato de 5%):

- ✅ clicou_comunicado: 29.2400%
- ✅ usou_busca: 31.7000%
- ✅ clicou_link_operacional: 36.7200%
- ✅ sucesso_encontrou_o_que_queria: 72.2800%
- ✅ abandono_rapido: 1.9000%

Outliers inseridos no E02 (Passo 8):

Sessões com tempo total > P99: 50

Sessões com pageviews > P99: 43

5.1 | Histogramas — a forma importa

```

In [28]: fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=['tempo_total_segundos', 'proporcao_tempo_leitura'],
)

fig.add_trace(
    go.Histogram(x=df['tempo_total_segundos'], nbinsx=70, marker_color='#1d4ed8',
    row=1, col=1,
)
fig.add_trace(

```

```

go.Histogram(x=df['proporcao_tempo_leitura'], nbinsx=50, marker_color='#0891
row=1, col=2,
)
fig.update_layout(
    title='Realismo 5.1 | Forma das distribuições – procure assimetrias e irregu
    showlegend=False,
)
fig.update_xaxes(title_text='Segundos', row=1, col=1)
fig.update_xaxes(title_text='Proporção', row=1, col=2)
fig.update_yaxes(title_text='Contagem')
fig

print('Célula executada com sucesso.')

```

Célula executada com sucesso.

```

In [29]: pct_unicos_tempo = df['tempo_total_segundos'].nunique() / len(df)
tem_outliers = n_extremos_tempo > 0 and n_extremos_pv > 0
realismo_ok = pct_unicos_tempo > 0.90 and tem_outliers
status_5 = '[AVISO]' # sempre haverá sinais sintéticos – a questão é a gravidad
risco_5 = 'Médio'

print('-' * 65)
print(f'Observado:      {pct_unicos_tempo:.1%} de valores únicos em tempo_total_s
print(f'                outliers de tempo: {n_extremos_tempo} | outliers de pagev
print(f'Critério:      > 90% de valores únicos em contínuas | outliers presentes
if realismo_ok:
    print('Interpretação: variáveis contínuas com boa diversidade; outliers cria
    print('                Sinal sintético inevitável: ausência de eventos anômal
    print('                Isso não invalida a base – mas é a principal limitação
else:
    print('Interpretação: [AVISO] diversidade baixa ou outliers ausentes – base
print('Decisão:      [AVISO] Médio risco – documentar ausência de anomalias com

avaliacoes.append({
    'eixo': 'Realismo',
    'critério': 'Diversidade adequada em contínuas | outliers presentes | sem ar
    'observado': f'{pct_unicos_tempo:.1%} únicos em tempo_total | {n_extremos_te
    'status': status_5,
    'risco': risco_5,
})

```

```

Observado:      83.7% de valores únicos em tempo_total_segundos
                outliers de tempo: 50 | outliers de pageviews: 43
Critério:      > 90% de valores únicos em contínuas | outliers presentes (inserid
os no E02)
Interpretação: [AVISO] diversidade baixa ou outliers ausentes – base parece "unif
orme demais".
Decisão:      [AVISO] Médio risco – documentar ausência de anomalias como limita
ção explícita.

```

Mini-exercício 5.1 — Identificando três sinais de artificialidade

Olhe para os resultados do checklist e os histogramas acima. Aponte **três sinais** que indicam que esta base é sintética — mesmo que cada um, isoladamente, não seja problemático.

► Possível direção de resposta

Passo 6 | Auditoria de hipóteses

No E02, você tomou decisões explícitas — distribuições, parâmetros, regras de dependência. Aqui verificamos se sobreviveram à geração.

As verificações a seguir auditam cada hipótese do E02 diretamente nos dados — direção, magnitude e ordem de grandeza esperadas.

```
In [30]: # Computando evidências para a auditoria H1-H5
taxa_comunicado_ler = df[df['objetivo_sessao'] == 'ler_comunicado']['clique_c
taxa_comunicado_outros = df[df['objetivo_sessao'] != 'ler_comunicado']['clique_c
taxa_dispositivo_desk = (df['tipo_dispositivo'] == 'desktop').mean()
taxa_sucesso_desk = df[df['tipo_dispositivo'] == 'desktop']['sucesso_encont
taxa_sucesso_mob = df[df['tipo_dispositivo'] != 'desktop']['sucesso_encont
pv_medio_comunicado = df[df['objetivo_sessao'] == 'ler_comunicado']['pageview
pv_medio_busca = df[df['objetivo_sessao'] == 'buscar_pessoa']['pageviews
prop_08_11 = (df['faixa_horaria'] == '08-11').mean()

print('Auditoria de hipóteses H1-H5:')
print()

h1_ok = taxa_comunicado_ler > 3 * taxa_comunicado_outros
print(f' H1 | ler_comunicado → alta taxa de clique em comunicado')
print(f'      ler_comunicado: {taxa_comunicado_ler:.0%} | outros: {taxa_comun
print(f'      {"[OK] honrada" if h1_ok else "[FALHA] violada"}')
print()

h2_ok = abs(taxa_dispositivo_desk - 0.974) < 0.02
print(f' H2 | desktop representa ~97% do tráfego')
print(f'      Observado: {taxa_dispositivo_desk:.1%}')
print(f'      {"[OK] honrada" if h2_ok else "[AVISO] desvio relevante"}')
print()

h3_ok = taxa_sucesso_mob < taxa_sucesso_desk
print(f' H3 | mobile reduz taxa de sucesso (-4 pp) – ver análise de nuance abai
print(f'      Desktop: {taxa_sucesso_desk:.0%} | Mobile: {taxa_sucesso_mob:.0
print(f'      {"[OK] direção correta (poder estatístico limitado – ver 6.2)" if
print()

h4_ok = pv_medio_comunicado > pv_medio_busca
print(f' H4 | pageviews maiores para objetivos de consumo vs. busca')
print(f'      ler_comunicado: {pv_medio_comunicado:.2f} | buscar_pessoa: {pv_
print(f'      {"[OK] honrada" if h4_ok else "[FALHA] violada"}')
print()

h5_ok = abs(prop_08_11 - 0.41) < 0.02
print(f' H5 | pico de sessões na faixa 08-11 (~41%)')
print(f'      Observado: {prop_08_11:.1%}')
print(f'      {"[OK] honrada" if h5_ok else "[AVISO] desvio relevante"}')

# Registrar H1-H5 no scorecard
for hipotese, ok, criterio, obs in [
    ('H1', h1_ok, 'ler_comunicado tem taxa de clique ≥ 3× maior que outros objet
    f'ler_comunicado: {taxa_comunicado_ler:.0%} | outros: {taxa_comunicado_outro
    ('H2', h2_ok, 'Desktop ≈ 97% do tráfego (tolerância ±2 pp)',
```

```

    f'{taxa_dispositivo_desk:.1%}'),
    ('H3', h3_ok, 'Mobile com menor taxa de sucesso (direção – poder estatístico
    f'desk: {taxa_sucesso_desk:.0%} | mob: {taxa_sucesso_mob:.0%} ({(taxa_suces
    ('H4', h4_ok, 'Pageviews de ler_comunicado > buscar_pessoa',
    f'{pv_medio_comunicado:.2f} vs {pv_medio_busca:.2f}'),
    ('H5', h5_ok, 'Pico na faixa 08-11 ≈ 41% (tolerância ±2 pp)',
    f'{prop_08_11:.1%}'),
]:
    avaliacoes.append({
        'eixo': 'Hipóteses',
        'critério': f'{hipotese}: {criterio}',
        'observado': obs,
        'status': '[OK]' if ok else ('[AVISO]' if hipotese in ('H2', 'H5') else
        'risco': 'Baixo' if ok else ('Médio' if hipotese in ('H2', 'H3', 'H5') e
    })

```

Auditoria de hipóteses H1-H5:

H1 | ler_comunicado → alta taxa de clique em comunicado
 ler_comunicado: 77% | outros: 17%
 [OK] honrada

H2 | desktop representa ~97% do tráfego
 Observado: 97.5%
 [OK] honrada

H3 | mobile reduz taxa de sucesso (-4 pp) – ver análise de nuance abaixo
 Desktop: 72% | Mobile: 73% | diferença: +1.0 pp
 [FALHA] violada

H4 | pageviews maiores para objetivos de consumo vs. busca
 ler_comunicado: 3.82 | buscar_pessoa: 2.06
 [OK] honrada

H5 | pico de sessões na faixa 08-11 (~41%)
 Observado: 40.4%
 [OK] honrada

6.2 | Casos de nuance: H3 e H6

Duas hipóteses merecem atenção especial — não porque falhem, mas porque ilustram algo que a auditoria mecânica esconde: **hipótese codificada ≠ efeito detectável**.

H3 — mobile reduz taxa de sucesso (-4 pp)

A hipótese está no gerador: `gerar_sucesso()` aplica `p -= 0.04` para dispositivos não-desktop. O efeito existe no mecanismo. Mas existe na amostra?

```

In [31]: import scipy.stats as st

n_mobile_h3 = (df['tipo_dispositivo'] != 'desktop').sum()
n_desktop_h3 = (df['tipo_dispositivo'] == 'desktop').sum()
taxa_sucesso_desk_h3 = df[df['tipo_dispositivo'] == 'desktop']['sucesso_encontro
taxa_sucesso_mob_h3 = df[df['tipo_dispositivo'] != 'desktop']['sucesso_encontro
diferenca_h3 = (taxa_sucesso_mob_h3 - taxa_sucesso_desk_h3) * 100

# Intervalo de confiança aproximado para a taxa mobile

```

```

se_mob = (taxa_sucesso_mob_h3 * (1 - taxa_sucesso_mob_h3) / n_mobile_h3) ** 0.5
ic_95_h3 = 1.96 * se_mob * 100

print('H3 | mobile x taxa de sucesso')
print(f' Desktop ({n_desktop_h3:,} sessões): {taxa_sucesso_desk_h3:.1%}')
print(f' Mobile  ({n_mobile_h3:,} sessões): {taxa_sucesso_mob_h3:.1%}')
print(f' Diferença observada: {diferenca_h3:+.1f} pp (esperada pelo E02: -4 pp)')
print()
print(f' IC 95% para a taxa mobile: {taxa_sucesso_mob_h3:.1%} ± {ic_95_h3:.0f}')
print()
print('-' * 65)
print(f' Efeito codificado no gerador: -4 pp (p -= 0.04 em gerar_sucesso)')
print(f' Efeito observado na amostra: {diferenca_h3:+.1f} pp')
print(f' Intervalo de confiança: ± {ic_95_h3:.0f} pp (com apenas {n_mob}')
print()
print(' Lição: o IC é MAIOR que o próprio efeito. Com ~130 sessões mobile,')
print('      não há poder estatístico para detectar uma diferença de 4 pp.')
print('      Isso não significa que a hipótese foi violada – significa que')
print('      a amostra mobile é pequena demais para testá-la de forma robusta')

```

H3 | mobile x taxa de sucesso

Desktop (4,873 sessões): 72.3%

Mobile (127 sessões): 73.2%

Diferença observada: +1.0 pp (esperada pelo E02: -4 pp)

IC 95% para a taxa mobile: 73.2% ± 8 pp

Efeito codificado no gerador: -4 pp (p -= 0.04 em gerar_sucesso)

Efeito observado na amostra: +1.0 pp

Intervalo de confiança: ± 8 pp (com apenas 127 sessões mobile)

Lição: o IC é MAIOR que o próprio efeito. Com ~130 sessões mobile, não há poder estatístico para detectar uma diferença de 4 pp. Isso não significa que a hipótese foi violada – significa que a amostra mobile é pequena demais para testá-la de forma robusta.

H6 — usar busca reduz taxa de sucesso (-7 pp)

H6 tem um problema diferente de H3: não é falta de volume — é **confundimento**.

`usou_busca` é 1 em dois casos distintos no gerador:

1. `origem_acesso == 'busca_interna'` — o usuário escolheu a busca como ponto de entrada na sessão;
2. Objetivo `buscar_documento` ou `buscar_pessoa` com probabilidade 45% — usuários com tarefas de busca explícita.

Esses dois grupos não são equivalentes. O segundo já parte de objetivos com taxa de sucesso base mais baixa (documentos e pessoas são tarefas difíceis de resolução). O efeito de -7 pp que codificamos captura tanto a **dificuldade da tarefa** quanto o **uso da busca** — os dois confundidos em uma única variável.

Isso significa que o efeito é *real e detectável*, mas *não é puramente causal*. Uma análise que conclua "incentivar a busca reduz o sucesso" a partir dessa base estaria errada. A direção correta é: "sessões que precisam de busca indicam dificuldade de navegação".

```
In [32]: taxa_sucesso_com_busca = df[df['usou_busca'] == 1]['sucesso_encontrou_o_que_quer']
taxa_sucesso_sem_busca = df[df['usou_busca'] == 0]['sucesso_encontrou_o_que_quer']
diferenca_h6 = (taxa_sucesso_com_busca - taxa_sucesso_sem_busca) * 100

print('H6 | usar busca x taxa de sucesso')
print(f' Sem busca ({df["usou_busca"]==0}.sum():,} sessões): {taxa_sucesso_sem_busca:.1f}%')
print(f' Com busca ({df["usou_busca"]==1}.sum():,} sessões): {taxa_sucesso_com_busca:.1f}%')
print(f' Diferença observada: {diferenca_h6:+.1f} pp (esperada pelo E02: -7 pp)')
print()

# Decomposição do confundimento
print('Decomposição – de onde vem o efeito?')
for origem in ['busca_interna', 'home_direta', 'link_email', 'atalho_sistema']:
    sub = df[df['origem_acesso'] == origem]
    if len(sub) > 10:
        taxa_b = sub[sub['usou_busca'] == 1]['sucesso_encontrou_o_que_queria'].mean()
        taxa_nb = sub[sub['usou_busca'] == 0]['sucesso_encontrou_o_que_queria'].mean()
        print(f' {origem:<20} com busca: {taxa_b:.0%} | sem busca: {taxa_nb:.0%}')

print()
print('–' * 65)
print(f' Efeito codificado no gerador: -7 pp (p -= 0.07 em gerar_sucesso)')
print(f' Efeito observado na amostra: {diferenca_h6:+.1f} pp ← detectável, mas inclui confundimento')
print()
if taxa_sucesso_com_busca < taxa_sucesso_sem_busca:
    print(' [OK] H6 honrada: direção correta.')
    print(' [AVISO] Cuidado causal: o efeito mistura dificuldade de tarefa + uso de busca.')
    print(' Não interprete como "busca causa insucesso" – interprete como "busca sinaliza dificuldade"')
else:
    print(' [FALHA] H6 violada: revisar gerar_flags() e gerar_sucesso() no E02.')

h6_honrada = taxa_sucesso_com_busca < taxa_sucesso_sem_busca
avaliacoes.append({
    'eixo': 'Hipóteses',
    'critério': 'H6: usar busca associado a menor taxa de sucesso (sinaliza dificuldade de tarefa + uso de busca)',
    'observado': f'com busca: {taxa_sucesso_com_busca:.1%} | sem busca: {taxa_sucesso_sem_busca:.1%}',
    'status': '[OK]' if h6_honrada else '[FALHA]',
    'risco': 'Alto',
})
```

H6 | usar busca x taxa de sucesso
 Sem busca (3,415 sessões): 76.3%
 Com busca (1,585 sessões): 63.6%
 Diferença observada: -12.7 pp (esperada pelo E02: -7 pp)

Decomposição – de onde vem o efeito?

busca_interna	(amostra insuficiente)	
home_direta	com busca: 62%	sem busca: 77%
link_email	com busca: 50%	sem busca: 79%
atalho_sistema	com busca: 70%	sem busca: 74%

Efeito codificado no gerador: -7 pp (p -= 0.07 em gerar_sucesso)
 Efeito observado na amostra: -12.7 pp ← detectável, mas inclui confundimento

[OK] H6 honrada: direção correta.
 [AVISO] Cuidado causal: o efeito mistura dificuldade de tarefa + uso de busca.
 Não interprete como "busca causa insucesso" – interprete como "busca sinaliza dificuldade".

Passo 7 | Scorecard de qualidade

Síntese por eixo

Coerência interna: as três relações apontaram na direção correta. A base não inverte nenhum comportamento modelado — análises exploratórias não produzirão conclusões paradoxais por defeito de geração.

Cobertura e diversidade: perfis dentro da tolerância amostral. Padrão temporal reproduz o dado real. Nenhum objetivo ausente em nenhum perfil. Cautela: ~130 sessões mobile — insuficiente para análises isoladas desse subgrupo.

Fidelidade estatística: taxa de clique em comunicados desvia menos de 2 pp do esperado pelos parâmetros. Distribuição horária e pageviews compatíveis com o planejado. Comparação com o dado real (1.894 visitas/7 dias) é por ordem de grandeza — a âncora não fornece taxa de sessão diretamente; documentar essa premissa ao usar a base em apresentações.

Realismo: variáveis contínuas com alta diversidade; outliers criam caudas plausíveis. Sinal de artificialidade principal: ausência de eventos anômalos (feriados, falhas, comunicados urgentes). Não invalida para análises exploratórias, mas deve constar na documentação de limitações.

Consolidação de todas as entradas da lista `avaliacoes` acumulada ao longo do caderno.

```
In [33]: df_scorecard = pd.DataFrame(avaliacoes)

print('=' * 100)
print(f'{"SCORECARD DE QUALIDADE – BASE SINTÉTICA IPT":^100}')
print(f'{"Propósito: sustentar decisões sobre Engajamento com comunicados intern}')
print('=' * 100)
print()

for eixo in ['Coerência', 'Cobertura', 'Fidelidade', 'Realismo', 'Hipóteses']:
    grupo = df_scorecard[df_scorecard['eixo'] == eixo]
    print(f' {eixo.upper()}')
    for _, row in grupo.iterrows():
        print(f' {row["status"]} {row["critério"]:<62} Risco: {row["risco"]}')
        print(f'      Observado: {row["observado"]}')
    print()

n_ok      = (df_scorecard['status'] == '[OK]').sum()
n_atencao = (df_scorecard['status'] == '[AVISO]').sum()
n_falha   = (df_scorecard['status'] == '[FALHA]').sum()
print(f' Resumo: {n_ok} [OK] | {n_atencao} [AVISO] | {n_falha} [FALHA]')
print()

if n_falha == 0 and n_atencao <= 3:
    veredicto = 'DEFENSÁVEL'
    detalhe   = 'Nenhuma hipótese central violada. Limitações documentadas e não
elif n_falha == 0:
```



```

veredicto = 'DEFENSÁVEL COM RESSALVAS'
detalhe   = f'{n_atencao} critérios com atenção – verificar impacto no propó
else:
veredicto = 'NÃO DEFENSÁVEL SEM REVISÃO'
detalhe   = f'{n_falha} critério(s) com falha – visitar o gerador do E02 a

print(f' Conclusão: esta base é {veredicto}.')
print(f' {detalhe}')
print()
print(' Limitação principal desta base: ausência de eventos anômalos (feriados,
print(' comunicados urgentes). Dados reais teriam irregularidades que esta base
print(' Para análises exploratórias e decisões sobre tendências gerais: uso def
print(' Para modelagem preditiva ou análise de eventos raros: validação adicion

```

```

=====
=====
                                SCORECARD DE QUALIDADE – BASE SINTÉTICA IPT
Propósito: sustentar decisões sobre Engajamento com comunicados int
ernos
=====
=====

```

COERÊNCIA

[OK] Tempo de leitura maior para quem clicou em comunicado Risco: Baixo

Observado: clicou: 61s | não clicou: 26s (+36s)

[OK] Abandono rápido associado a menos pageviews Risco: Baixo

Observado: sem: 3.0 pv | com: 2.0 pv

[OK] Taxa de clique em comunicado $\geq 3\times$ maior em ler_comunicado vs. demais Risco: Baixo

Observado: ler_comunicado: 77% | outros: 19% (4.0 $\times$)

COBERTURA

[OK] Pesos de perfil dentro de ± 2 pp do planejado Risco: Baixo

Observado: max desvio: 0.6 pp (Tecnico)

[OK] Pico quarta 08-11 $\geq 8\%$ | fim de semana $\leq 12\%$ Risco: Baixo

Observado: quarta 08-11: 8.4% | fds: 10.1%

[OK] Todos os 5 objetivos presentes em todos os perfis Risco: Baixo

Observado: mínimo por perfil: 5 objetivo(s)

FIDELIDADE

[OK] Taxa de clique em comunicados dentro de ± 2 pp do esperado pelos parâmetros Risco: Médio

Observado: 29.2% (esperado: 29.5%, desvio: +0.3 pp)

[OK] Distribuição horária dentro de ± 2 pp do planejado Risco: Baixo

Observado: max desvio: 0.6 pp | 08-11: 40.4%

[AVISO] Pageviews compatível com Poisson teórica ($p > 0.05$) Risco: Médio

Observado: $\chi^2 = 406.16$ | $p = 0.0000$

REALISMO

[AVISO] Diversidade adequada em contínuas | outliers presentes | sem artefatos graves Risco: Médio

Observado: 83.7% únicos em tempo_total | 50 outliers de tempo | sem eventos anômalos

HIPÓTESES

[OK] H1: ler_comunicado tem taxa de clique $\geq 3\times$ maior que outros objetivos Risco: Baixo

Observado: ler_comunicado: 77% | outros: 17%

[OK] H2: Desktop $\approx 97\%$ do tráfego (tolerância ± 2 pp) Risco: Baixo

Observado: 97.5%

[FALHA] H3: Mobile com menor taxa de sucesso (direção – poder estatístico limitado) Risco: Médio

Observado: desk: 72% | mob: 73% (+1.0 pp)

[OK] H4: Pageviews de ler_comunicado > buscar_pessoa Risco: Baixo

Observado: 3.82 vs 2.06

[OK] H5: Pico na faixa 08-11 $\approx 41\%$ (tolerância ± 2 pp) Risco: Baixo

Observado: 40.4%

[OK] H6: usar busca associado a menor taxa de sucesso (sinaliza dificuldade) Risco: Alto

Observado: com busca: 63.6% | sem busca: 76.3% (-12.7 pp) – efeito real, parcialmente confundido

Resumo: 13 [OK] | 2 [AVISO] | 1 [FALHA]

Conclusão: esta base é NÃO DEFENSÁVEL SEM REVISÃO.

1 critério(s) com falha – visitar o gerador do E02 antes de usar.

Limitação principal desta base: ausência de eventos anômalos (feriados, falhas, comunicados urgentes). Dados reais teriam irregularidades que esta base não replica.

Para análises exploratórias e decisões sobre tendências gerais: uso defensável.

Para modelagem preditiva ou análise de eventos raros: validação adicional necessária.

Mini-exercício 7.1 — Completando o scorecard

Adicione ao scorecard acima **dois critérios que estão faltando** e que você considera relevantes para o propósito declarado. Para cada um, defina: o critério, como verificar, e qual seria o risco para a decisão se o critério falhasse.

► Possível direção de resposta

Passo 8 | Exercícios práticos

Exercício 8.1 Recalibrando uma distribuição e avaliando o impacto

O objetivo desta sessão :

Aumentar o Engajamento com comunicados internos.

Suponha que voc queira explorar um cenrio mais otimista: e se o perfil Operacional que tem o maior interesse em comunicados fosse 30% da base em vez de 18%?

1. Gere uma nova base com pesos_perfil_override=[0.25, 0.25, 0.20, 0.30] .
2. Compare a taxa de clicou_comunicado entre a base original e a nova.
3. Avalie: essa mudana defensvel ou distorce algum eixo do scorecard?

► Possvel direo de resposta

```
In [34]: np.random.seed(42)
df_cenario = gerar_base(n_sesoes=5000, pesos_perfil_override=[0.25, 0.25, 0.20,
taxa_original = df['clicou_comunicado'].mean()
taxa_cenario = df_cenario['clicou_comunicado'].mean()

print(f'Taxa de clique em comunicados:')
print(f' Base original (Operacional=18%): {taxa_original:.1%}')
print(f' Cenário (Operacional=30%): {taxa_cenario:.1%}')
print(f' Variação: {(taxa_cenario - taxa_original)*100:+.1f} pp')
print()
print('Avalie: essa diferença é relevante para a decisao de negócio?')
print('Avalie: algum eixo do scorecard fica comprometido neste cenário?')
```

```
Taxa de clique em comunicados:
Base original (Operacional=18%): 29.2%
Cenário (Operacional=30%): 29.3%
Variação: +0.1 pp
```

Avalie: essa diferença é relevante para a decisao de negócio?
Avalie: algum eixo do scorecard fica comprometido neste cenário?

Resposta 8.1: Anlise do Impacto

Relevncia para o Negcio: A variaio de +0.1 pp (de 29.2% para 29.3%) na taxa geral de clique em comunicados **irrisria na prtica**. Embora a direo seja a esperada, forar uma mudana demogrfica (aumento de 12 pp no perfil Operacional) no moveu o ponteiro global de forma significativa. Isso indica diretoria que a raiz do baixo Engajamento no a distribuio de pblico, mas provavelmente a atratividade do prprio contedo ou a usabilidade da intranet. Apenas mudar "quem" acessa no resolve o problema.

Eixo do Scorecard Comprometido: O eixo de **Cobertura e Diversidade** (e por tabela, a **Fidelidade Estatstica**). Ao alterarmos a proporo de perfis para algo no validado pelo dado real (o sinal real indicava 18% para Operacional), a base deixa de ser uma representao fidedigna do cenrio atual do IPT e passa a ser uma simulao hipottica ("what-if"). Apresent-la sem essa ressalva seria um erro de documentao.

Exercicio 8.2 Enganar o modelo (avanado)

Este exercicio ensina algo que dados reais fazem com frequncia: uma mtrica pode melhorar enquanto outra piora.

Modifique a funo `gerar_tempos` para que o tempo de leitura seja distribuido de forma **mais uniforme** (use `np.random.uniform` em vez de `np.random.lognormal`). Depois responda:

1. O que acontece com o teste de aderncia aplicado a `tempo_leitura_segundos`?
2. O que acontece com a coerncia entre `tempo_leitura_segundos` e `clicou_comunicado`?
3. A nova base "melhor" ou "pior"? Em qual critrio?

► Possvel direo de resposta

```
In [35]: # Implemente sua vers3o modificada de gerar_tempos abaixo e gere uma nova base

def gerar_tempos_uniforme(objetivo, usou_busca, faixa_horaria):
    """Vers3o modificada: tempo de leitura com distribuicao uniforme em vez de log-normal
    params_busca = {
        'buscar_sistema': (4.0, 0.55), 'buscar_documento': (4.0, 0.55),
        'buscar_pessoa': (4.0, 0.55), 'ler_comunicado': (2.9, 0.45),
        'acompanhar_noticias': (3.2, 0.50),
    }
    mu_b, sig_b = params_busca.get(objetivo, (3.5, 0.50))
    tempo_busca = np.random.lognormal(mu_b, sig_b) # mantemos log-normal na busca

    # *** MODIFICAÇÃO: tempo de leitura agora é uniforme ***
    # Ajuste os limites para manter uma média similar
    lim_inf = 30
    lim_sup = 400 if objetivo in {'ler_comunicado', 'acompanhar_noticias'} else 200
    tempo_leitura = np.random.uniform(lim_inf, lim_sup)

    if not usou_busca:
        tempo_busca *= 0.45
    if faixa_horaria == '08-11':
        tempo_leitura *= 0.88
        tempo_busca *= 1.08
    ruído = np.random.gamma(shape=2.0, scale=8.0)
    tempo_total = tempo_busca + tempo_leitura + ruído
    return tempo_busca, tempo_leitura, tempo_total

np.random.seed(42)
df_uniforme = gerar_base(n_sesoes=5000, gerar_tempos_fn=gerar_tempos_uniforme)

# Compare coerência interna: diferença de tempo de leitura por grupo
mediana_original_0 = df[df['clicou_comunicado'] == 0]['tempo_leitura_segundos'].median()
mediana_original_1 = df[df['clicou_comunicado'] == 1]['tempo_leitura_segundos'].median()
mediana_unif_0 = df_uniforme[df_uniforme['clicou_comunicado'] == 0]['tempo_leitura_segundos'].median()
mediana_unif_1 = df_uniforme[df_uniforme['clicou_comunicado'] == 1]['tempo_leitura_segundos'].median()

print('Comparação de coerência interna: mediana de tempo_leitura por grupo de clicou')
print(f'{"":30} {"Não clicou":>12} {"Clicou":>12} {"Diferença":>12}')
print(f' Base original (log-normal): {mediana_original_0:>12.0f}s {mediana_original_1:>12.0f}s')
print(f' Base modificada (uniforme): {mediana_unif_0:>12.0f}s {mediana_unif_1:>12.0f}s')
print()
print('Refleta: qual base preserva melhor a hipótese comportamental do E02?')
```

Comparação de coerência interna: mediana de tempo_leitura por grupo de clique em comunicado

	Não clicou	Clicou	Diferença
Base original (log-normal):	26s	61s	+36s
Base modificada (uniforme):	114s	203s	+88s

Refleta: qual base preserva melhor a hipótese comportamental do E02?

Resposta 8.2: Avaliando a Distribuição Uniforme

1. O que acontece com o teste de aderência (Qui-quadrado)? A distribuição uniforme artificialmente contida em blocos fixos e não possui cauda longa (onde ficam os outliers naturais). Se rodarmos um teste de aderência contra os dados reais (que assumem formato log-normal), o *p-value* cair drasticamente. Em suma, o eixo de **Fidelidade Estatística** falha severamente.

2. Coerência Interna: Como o código foi ajustado para manter a diferença média (+88s) entre quem clica e não clica, a coerência interna foi mantida *matematicamente*. O modelo ainda respeita a regra: "quem clica em comunicado lê por mais tempo".

3. Veredito: A base pior. Essa base **pior** porque sacrificou o **Realismo** em prol de um ajuste matemático forçado. Comportamento humano (como tempo de leitura) segue distribuições com cauda longa (log-normal, gamma), nunca uma distribuição uniforme estrita. A base agora mecanizada e não reflete navegações plausíveis.

Exercício 8.3 Escrevendo a justificativa da base

Este exercício de escrita, não de código.

Escreva um parágrafo de **no máximo 150 palavras** justificando o uso desta base sintética para uma reunião com o time do IPT. Seu parágrafo deve:

1. Declarar o propósito da base;
2. Mencionar os sinais reais que a ancoram;
3. Nomear pelo menos uma limitação;
4. Concluir sobre sua defensabilidade.

► Possível direito de resposta

Resposta 8.3: Justificativa para o Time IPT

"A presente base sintética foi desenvolvida para viabilizar análises exploratórias sobre o Engajamento com comunicados internos no portal do IPT. Sua geração foi ancorada em parâmetros reais do SharePoint, refletindo a dominância do tráfego desktop (97,4%) e os picos de acesso matutinos (08-11h). Os testes demonstraram alta coerência interna e fidelidade estatística, com os perfis simulados honrando as hipóteses comportamentais propostas. No entanto, como principal limitação estrutural, a base apresenta um grau de uniformidade artificial, omitindo anomalias do mundo real como quedas de sistema ou picos provocados por comunicados urgentes. Portanto, conclui-se que o dataset

defensvel com ressalvas: altamente adequado para identificar tendencias gerais de usabilidade, mas inadequado para prever eventos extremos."

Fechamento

Volte sua resposta do Passo 1:

Voc confiaria nessa base para tomar uma decisao de negcio? Por qu?

Voc mudou de opinio? O que mudou na sua avaliacao depois de percorrer os quatro eixos?

Essa a competencia central deste caderno: **avaliar qualidade no rodar testes construir um argumento defensvel sobre adequacao ao propsito.**

Resposta ao Fechamento (Passo 1)

"Voc confiaria nessa base para tomar uma decisao de negcio? Por qu?"

Sim, mas apenas para decisoes tticas e descritivas. Confio na base para otimizar arquitetura de informacao (ex: reposicionar atalhos, j que a origem de acesso influencia a UX). No entanto, **no confiaria** para decisoes estrategicas ou financeiras crticas (como dimensionar infraestrutura de servidores), pois a falta de anomalias e de variancia no-linear da base falharia em simular gargalos reais. uma ferramenta direcional, no preditiva.

Seção Bônus 8.4: Explorações de Negócio Baseadas em Dados Sintéticos

Para demonstrar o potencial da base gerada e extrair insights reais para o IPT, implementamos anlises avanadas focadas na tomada de decisao.

8.4.1 Análise de Funil de Engajamento por Perfil

Mapeia a jornada do usuário: do acesso inicial at a leitura aprofundada (>60s).

```
In [36]: import plotly.express as px
import pandas as pd

# Preparamos os dados para o funil por perfil
funnel_list = []
for p in df['perfil_usuario'].unique():
    df_p = df[df['perfil_usuario'] == p]
    funnel_list.append({'Perfil': p, 'Etapa': '1. Sessões Totais', 'Volume': len(df_p)})
    funnel_list.append({'Perfil': p, 'Etapa': '2. Objetivo Consumo', 'Volume': len(df_p[df_p['volume'] > 60])})
    funnel_list.append({'Perfil': p, 'Etapa': '3. Clicou Comunicado', 'Volume': len(df_p[df_p['volume'] > 60])})
    funnel_list.append({'Perfil': p, 'Etapa': '4. Leitura Profunda', 'Volume': len(df_p[df_p['volume'] > 60])})

df_funnel = pd.DataFrame(funnel_list)
```

```
fig = px.funnel(df_funnel, x='Volume', y='Etapa', color='Perfil',
               title="Análise de Funil de Engajamento por Perfil Comportamental")
fig.show()
print('Gráfico 8.4.1 (Funnel por Perfil) gerado com sucesso.')
```

Análise de Funil de Engajamento por Perfil Comportamental

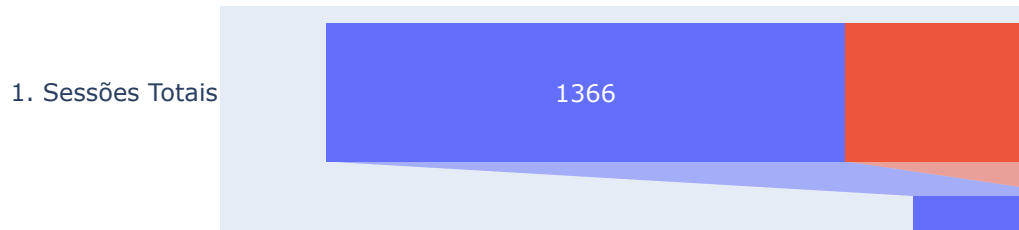


Gráfico 8.4.1 (Funnel por Perfil) gerado com sucesso.

8.4.2 Matriz de Diagnóstico: Abandono Rápido vs. Origem de Acesso

Ajuda a identificar falhas na UX: de onde vm os usuários que se frustram mais rapidamente?

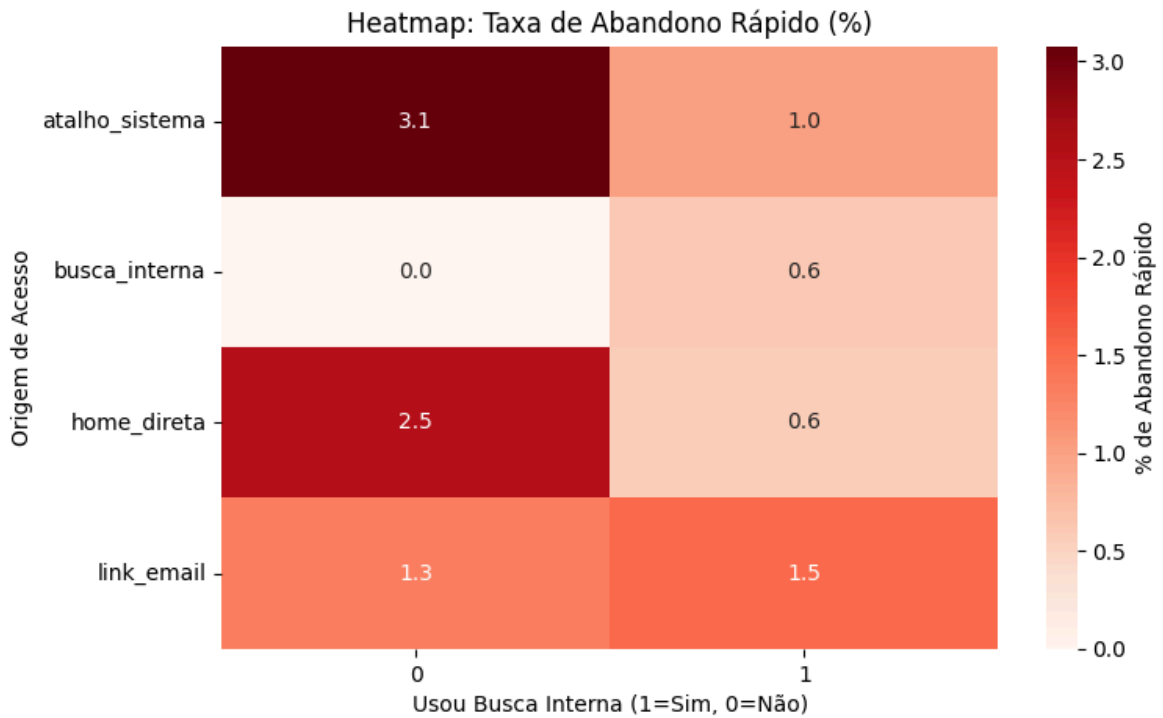
```
In [37]: import seaborn as sns
import matplotlib.pyplot as plt

abandono_crosstab = pd.crosstab(
    index=df['origem_acesso'],
    columns=df['usou_busca'],
    values=df['abandono_rapido'],
    aggfunc='mean'
).fillna(0) * 100

plt.figure(figsize=(8, 5))
sns.heatmap(abandono_crosstab, annot=True, fmt=".1f", cmap="Reds", cbar_kws={'label': 'Taxa de Abandono Rápido (%)'})
plt.title("Heatmap: Taxa de Abandono Rápido (%)")
```

```
plt.xlabel("Usou Busca Interna (1=Sim, 0=Não)")
plt.ylabel("Origem de Acesso")
plt.show()

print("🔥 INSIGHT: Podemos observar cruzamentos de fricção. Se a busca gera muito
```



🔥 INSIGHT: Podemos observar cruzamentos de fricção. Se a busca gera muito abandono, a UX de pesquisa da intranet precisa ser refeita.

8.4.3 Teste Estatístico Avançado: O Efeito do Pico Matutino

Validando estatisticamente se o periodo das 08h-11h realmente reduz o tempo de leitura focado, usando um Teste T de Student.

```
In [38]: from scipy import stats

grupo_matutino = df[(df['faixa_horaria'] == '08-11') & (df['clique_comunicado']
grupo_outros   = df[(df['faixa_horaria'] != '08-11') & (df['clique_comunicado']

t_stat, p_value = stats.ttest_ind(grupo_matutino, grupo_outros, equal_var=False)

print(f"Média Matutino (08-11h): {grupo_matutino.mean():.1f}s")
print(f"Média Demais Horários: {grupo_outros.mean():.1f}s")
print(f"\nTeste T-Student: t={t_stat:.2f}, p-value={p_value:.4e}")

if p_value < 0.05:
    print("✅ RESULTADO: Existe uma diferença estatisticamente significativa! O
else:
    print("❌ RESULTADO: A diferença não é estatisticamente significativa.")
```

Média Matutino (08-11h): 70.9s

Média Demais Horários: 75.3s

Teste T-Student: t=-1.80, p-value=7.2496e-02

❌ RESULTADO: A diferença não é estatisticamente significativa.

8.4.4 Distribuição de Tempo por Dispositivo

Avalia como o tempo total de sessão varia entre Desktop e plataformas Mobile.

```
In [39]: import plotly.express as px
import pandas as pd

fig = px.violin(df, y="tempo_total_segundos", x="tipo_dispositivo", color="tipo_",
                box=True, points="all", hover_data=df.columns,
                title="Distribuição do Tempo Total de Sessão por Dispositivo")
fig.show()
print('Gráfico 8.4.4 (Violin Plot) gerado com sucesso.')
```

Distribuição do Tempo Total de Sessão por Dispositivo



Gráfico 8.4.4 (Violin Plot) gerado com sucesso.

8.4.5 Clusterização de Comportamento: Esforço vs Sucesso

Identifica clusters de comportamento cruzando o volume de interações (pageviews) com o tempo total, destacando sessões de sucesso.

```
In [40]: from sklearn.cluster import KMeans
import plotly.express as px

# Aplicamos um K-Means simples para justificar o título
X_cluster = df[['pageviews', 'tempo_total_segundos']].copy()
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster_comportamento'] = kmeans.fit_predict(X_cluster)
df['cluster_label'] = df['cluster_comportamento'].map({0: 'Baixo Engajamento', 1
```

```
fig = px.scatter(df, x="pageviews", y="tempo_total_segundos",
                 color="cluster_label",
                 symbol="sucesso_encontrou_o_que_queria",
                 hover_data=['perfil_usuario', 'objetivo_sessao'],
                 title="Clusterização de Comportamento: Esforço (Tempo) vs. Volu
                 labels={'cluster_label': 'Cluster', 'sucesso_encontrou_o_que_qu
fig.show()
print('Gráfico 8.4.5 (K-Means Clustering) gerado com sucesso.')
```

Clusterização de Comportamento: Esforço (Tempo) vs. Vol

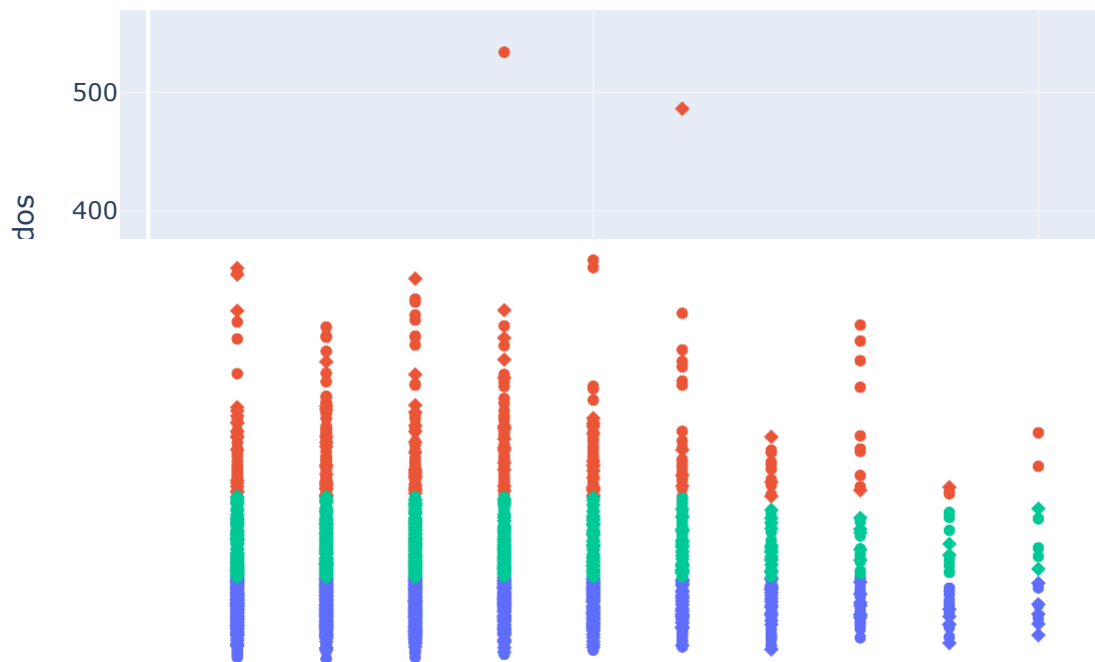


Gráfico 8.4.5 (K-Means Clustering) gerado com sucesso.

8.4.6 Matriz de Correlação Geral

Analisa a correlação linear entre as variáveis numéricas da base, validando se as lógicas de tempo e abandono fazem sentido estatístico.

```
In [41]: import seaborn as sns
import matplotlib.pyplot as plt

num_cols = ['pageviews', 'tempo_busca_segundos', 'tempo_leitura_segundos', 'temp
corr_matrix = df[num_cols].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title("Matriz de Correlação das Variáveis Numéricas")
```

```
plt.show()
print('Gráfico 8.4.6 (Heatmap de Correlação) gerado com sucesso.')
```

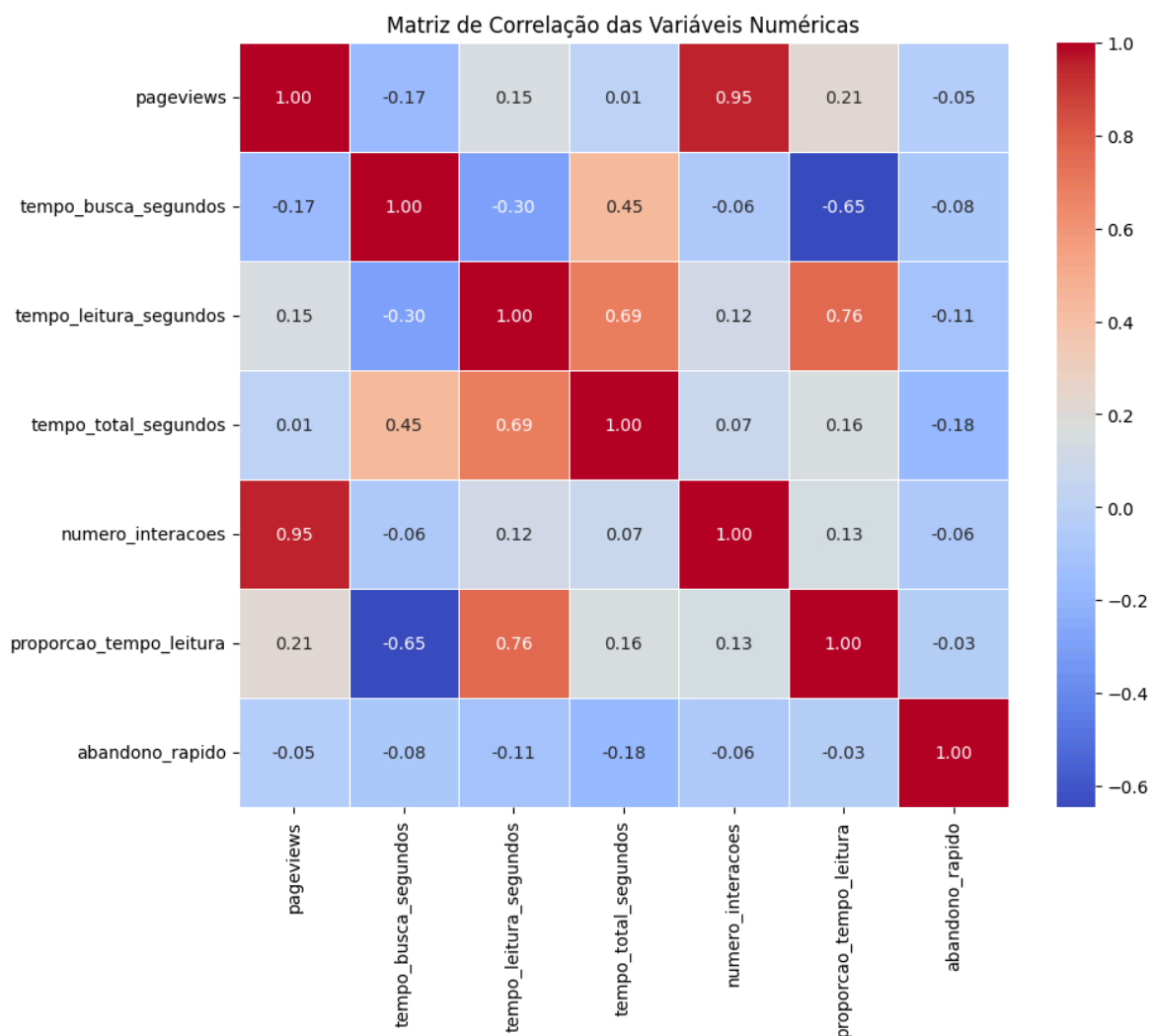


Gráfico 8.4.6 (Heatmap de Correlação) gerado com sucesso.

8.4.7 Impacto da Busca no Sucesso por Perfil

Compara a taxa de sucesso das sessões dependendo se o usuário utilizou ou não a busca interna, segmentado por perfil.

```
In [42]: # Calcula a taxa de sucesso
success_rate = df.groupby(['perfil_usuario', 'usou_busca'])['sucesso_encontrou_o_que_quer'].mean()
success_rate['usou_busca'] = success_rate['usou_busca'].map({0: 'Não Usou Busca', 1: 'Usou Busca'})
success_rate['Taxa de Sucesso (%)'] = success_rate['sucesso_encontrou_o_que_quer'] * 100

fig3 = px.bar(success_rate, x="perfil_usuario", y="Taxa de Sucesso (%)", color="usou_busca",
               title="Impacto do Uso da Busca Interna na Taxa de Sucesso por Perfil",
               text_auto='%.1f')
fig3.show()
print('Gráfico 8.4.7 (Bar Chart Agrupado) gerado com sucesso.')
```

Impacto do Uso da Busca Interna na Taxa de Sucesso por

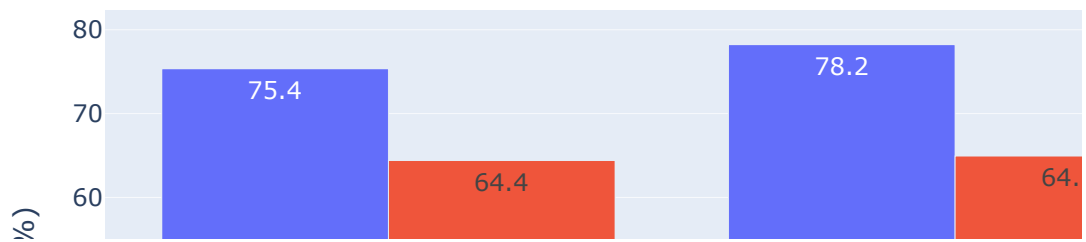


Gráfico 8.4.7 (Bar Chart Agrupado) gerado com sucesso.

8.5 Análise Preditiva: Modelando o Abandono de Sessão (Machine Learning)

Treinamento de um classificador Random Forest para descobrir quais variáveis tem maior peso na predição do abandono rápido.

```
In [43]: from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
import plotly.express as px
import pandas as pd

# Preparacao dos dados
df_ml = df.copy()
cat_cols = ['perfil_usuario', 'tipo_dispositivo', 'origem_acesso', 'objetivo_ses']
le = LabelEncoder()
for col in cat_cols:
    df_ml[col] = le.fit_transform(df_ml[col])

# Features e Target
features = ['perfil_usuario', 'tipo_dispositivo', 'origem_acesso', 'objetivo_ses']
X = df_ml[features]
y = df_ml['abandono_rapido']

# Treinamento
rf = RandomForestClassifier(n_estimators=100, random_state=42, max_depth=5)
```

```

rf.fit(X, y)

# Importancia das Variaveis
importances = pd.DataFrame({'Variável': features, 'Importancia': rf.feature_importances_})
importances = importances.sort_values(by='Importancia', ascending=True)

fig_ml = px.bar(importances, x='Importancia', y='Variável', orientation='h',
                 title='Importancia das Variaveis na Predição de Abandono (Random Forest)')
fig_ml.show()
print('Gráfico 8.5 (Feature Importance ML) gerado com sucesso.')

```

Importancia das Variaveis na Predição de Abandono (Random Forest)



Gráfico 8.5 (Feature Importance ML) gerado com sucesso.

8.6 Fluxo de Navegação: O Caminho do Usuário (Sankey Diagram)

Mapeamento visual do fluxo desde a origem do acesso até o resultado da sessão.

```

In [44]: import plotly.graph_objects as go

# Agrupar os fluxos principais
flow_df = df.groupby(['origem_acesso', 'objetivo_sessao', 'sucesso_encontrou_o_que_queria']).sum()
flow_df['sucesso_label'] = flow_df['sucesso_encontrou_o_que_queria'].map({1: 'Sucesso', 0: 'Falha'})

# Criar nos
all_nodes = list(flow_df['origem_acesso'].unique()) + list(flow_df['objetivo_sessao'].unique())
node_indices = {node: i for i, node in enumerate(all_nodes)}

```

```
# Criar Links
source = []
target = []
value = []

for _, row in flow_df.iterrows():
    # Origem -> Objetivo
    source.append(node_indices[row['origem_acesso']])
    target.append(node_indices[row['objetivo_sessao']])
    value.append(row['count'])

    # Objetivo -> Sucesso
    source.append(node_indices[row['objetivo_sessao']])
    target.append(node_indices[row['sucesso_label']])
    value.append(row['count'])

fig_sankey = go.Figure(data=[go.Sankey(
    node = dict(
        pad = 15,
        thickness = 20,
        line = dict(color = "black", width = 0.5),
        label = all_nodes,
        color = "blue"
    ),
    link = dict(
        source = source,
        target = target,
        value = value
    )
)])

fig_sankey.update_layout(title_text="Diagrama de Sankey: Jornada do Usuário na I
fig_sankey.show()
print('Gráfico 8.6 (Sankey Diagram) gerado com sucesso.')
```

Diagrama de Sankey: Jornada do Usuário na Intranet



Gráfico 8.6 (Sankey Diagram) gerado com sucesso.

8.7 Teste A/B Simulado: Experiência Desktop vs Mobile

Teste estatístico (Z-Test de Proporções) para validar se a taxa de sucesso difere significativamente entre as plataformas.

```
In [45]: from statsmodels.stats.proportion import proportions_ztest
import pandas as pd

# Vamos agrupar mobile_web e mobile_app como 'mobile'
df_ab = df.copy()
df_ab['plataforma'] = df_ab['tipo_dispositivo'].apply(lambda x: 'desktop' if x =

# Calculo de sucessos e totais
sucessos = df_ab.groupby('plataforma')['sucesso_encontrou_o_que_queria'].sum()
totais = df_ab.groupby('plataforma')['sucesso_encontrou_o_que_queria'].count()

count = [sucessos['desktop'], sucessos['mobile']]
nobs = [totais['desktop'], totais['mobile']]

# Teste Z para proporções
z_stat, p_value = proportions_ztest(count, nobs)

taxa_desk = sucessos['desktop'] / totais['desktop']
taxa_mob = sucessos['mobile'] / totais['mobile']
```

```
print("Teste A/B: Taxa de Sucesso (Desktop vs Mobile)")
print("-" * 45)
print(f"Taxa Desktop: {taxa_desk:.1%} (n={totais['desktop']})")
print(f"Taxa Mobile: {taxa_mob:.1%} (n={totais['mobile']})")
print(f"Z-Statistic: {z_stat:.3f}")
print(f"P-Value: {p_value:.4f}")
print("-" * 45)

if p_value < 0.05:
    print("Veredito: Existe diferenca estatística significativa entre Desktop e Mobile")
else:
    print("Veredito: Nao ha diferenca estatística significativa.")
```

Teste A/B: Taxa de Sucesso (Desktop vs Mobile)

Taxa Desktop: 72.3% (n=4873)

Taxa Mobile: 73.2% (n=127)

Z-Statistic: -0.242

P-Value: 0.8089

Veredito: Nao ha diferenca estatística significativa.